

รายงาน Transform: กฎทำงานยังไง และทำให้เป็น White Box

งานนี้ตอบอะไร

โจทย์จากเพื่อน (คัดลอกตามที่ได้รับ):

Transform ส่วนที่ 1 อธิบายกฎ ต้องตอบ 4 ข้อ -ตั้งกฎไปทำไม แต่ละกฎกันปัญหาอะไร -ผู้ใช้นำข้อมูลเข้าแล้วตั้งค่ากฎ ค่านั้นถูกส่งไปไหน -ตัวเลขในกฎถูกเอาไปคำนวณยังไง ตัดสินว่า
แถวไหนผ่านหรือไม่ผ่าน -หลังตรวจสอบกฎแล้ว ข้อมูลไปต่อที่กระบวนการไหน

ส่วนที่ 2 วิเคราะห์ Transform ว่าควรปรับอะไร และควรเน้นอะไร

ข้อกำหนดเพิ่มจากเพื่อน 2 ข้อ:

- "กฎ" หมายถึงทั้งสองชุด: การ์ด 3 ใบบนหน้า Rule (ช่วงคะแนน / ห้ามซ้ำ / ค่าผิดปกติ) และ กฎรายการในตั้งค่าขั้นสูง ("รวมเลย")
- ส่วนที่ 2 ต้องหาจุด black box ในหน้าที่เกี่ยวกับ Transform และเสนอวิธีทำให้เป็น white box "โดยไม่ใช้วิธีการเขียนอธิบายเพิ่มหรือเนื้อหาให้อ่านลงในเว็บ"

รายงานนี้เป็น เอกสารอย่างเดียว ไม่มีการแก้ไขโค้ดของระบบ และไม่มีการเปลี่ยนค่าในระบบระหว่างเก็บหลักฐาน

อ่านตรงไหนก่อน

ไฟล์	เนื้อหา
01-interactive-rules.md	ส่วนที่ 1A: การ์ด 3 ใบ ตอบ 4 ข้อ พร้อมตัวอย่างตัวเลขจากการรันเอนจินจริง 10 สถานการณ์
02-spark-table-rules.md	ส่วนที่ 1B: กฎรายการ (Spark) ตอบ 4 ข้อ ตารางว่าช่องไหนตั้งแล้วมีผลจริง และเปรียบเทียบกับการ์ด
03-transform-analysis.md	ส่วนที่ 2: รายการ black box 30 จุด (BB-01 ถึง BB-30) วิธีทำเป็น white box ที่ไม่ใช่ข้อความ และลำดับความสำคัญ

คำตอบสั้น 4 ข้อ (ส่วนที่ 1)

ข้อ 1: ตั้งกฎไปทำไม แต่ละกฎกันปัญหาอะไร

- การ์ด: คะแนนที่เป็นไปไม่ได้/ว่าง, แถวซ้ำ, ค่าเรียนที่ผิดปกติ (รายละเอียด)

- Spark: ลบซ้ำ, คีย์หลักว่าง, ค่าว่างทุกคอลัมน์, ชนิดข้อมูล, วันที่ว่าง, ค่าผิดปกติ IQR และ Z-score (รายละเอียด)

ข้อ 2: ตั้งค่าแล้ว ค่าถูกส่งไปไหน

- การด: ส่งไป api ทันทีทุกครั้งที่แก้ เก็บในหน่วยความจำและไฟล์ แล้วคำนวณใหม่ทั้งไฟล์ ปุ่ม "บันทึกกฎ" แลส่งซ้ำ (รายละเอียด)
- Spark: บันทึกลง rules_config.json และ Elasticsearch ทันที แต่มีผลเฉพาะรอบที่รันครั้งถัดไป Spark อ่าน Elasticsearch ก่อนไฟล์ (รายละเอียด)

ข้อ 3: ตัวเลขในกฎถูกเอาไปคำนวณยังไง

- การด: ลำดับจริงคือ ซ้ำ → ว่าง/นอกช่วง → ค่าผิดปกติ (รั้ว Tukey = $Q3 + k \times IQR$) ตัวอย่าง: $Q1 = 3, Q3 = 8$, รั้วบน $k = 3.0$ คือ 23.0 ชั่วโมง (รายละเอียด)
- Spark: ลบซ้ำ → ตรวจแถวที่ละข้อ → IQR → Z-score แล้วเทียบคะแนนกับเกณฑ์ ตัวอย่าง: $grocery_sales \ 557/561 = 99.287\%$ เทียบเกณฑ์ 90.0 (รายละเอียด)

ข้อ 4: หลังตรวจแล้ว ข้อมูลไปที่ไหน

- การด: ติดป้าย 3 คอลัมน์แล้วแยกเป็น 3 ไฟล์ Clean / Review / Quarantine ไปที่หน้า Jobs & Pipelines และ Workspace Exports ไม่แก้ค่าในข้อมูล (รายละเอียด)
- Spark: แถวที่ผ่าน MERGE เข้าตาราง active สะสม แถวที่ไม่ผ่านต่อท้ายตาราง quarantine ตามรอบ คะแนนต่ำกว่าเกณฑ์แจ้งเตือน ตั้งแต่เกณฑ์ขึ้นไปเรียก Gold layer (รายละเอียด)

สรุปส่วนที่ 2 ใน 1 หน้าจอ

จุด black box: 30 จุด แบ่งเป็น Data Ingestion 5, การด 3 ใบ 11, กฎรายการ Spark 8, Jobs & Pipelines 5, Workspace Exports 1

ควรปรับอะไร (เรียงตามลำดับ)

ลำดับ	สิ่งที่ควรปรับ	ปิด BB
1	ใบบันทึกการรัน (manifest) ของทั้งสองเอนจิน แสดงค่าที่ใช้จริงเทียบกับค่าที่ตั้ง (ต้องซ่อม BB-18 พร้อมกัน)	02, 17, 19, 21, 23, 24, 27, 30
2	กราฟการกระจายพร้อมเส้นรั้วที่เลื่อนตาม k ได้	09, 16
3	ร่าง → เห็นผลต่าง → ยืนยัน พร้อมเลขเวอร์ชันชุดกฎ	06, 13, 14, 22
4	เหตุผลรายแถวแบบแยกช่อง + กดตัวเลขแล้วเห็นแถว	11, 25, 26, 29
5	เอาของปลอมออก (วงกลม 95/99/99, ป้าย > 12.0h, ของไม่มีผล 5 ช่อง, ปุ่มเชื่อมต่อที่ไม่ได้ต่อ, สถานะสำเร็จที่ไม่จริง)	03, 07, 08, 20
6	แสดงคีย์และชุดข้อมูลที่ใช้จริง + จับคู่คอลัมน์ตอนอัปเดต	01, 04, 12, 15
7	แผนภาพไหลตามลำดับตรวจจริง	10, 28
8	รายการชุดข้อมูลที่นำเข้า	05

ควรเน้นอะไร: (1) ตามรอยได้ตลอดสาย ค่ากฎ → ตัวเลข → แถว → ไฟล์ (2) ตัวเลขหนึ่งตัว มีแหล่งเดียว (3) ไม่มีช่องที่ตั้งแล้วไม่มีผล (4) สองเอนจินที่พฤติกรรมต่างกัน ต้องแสดงตัวตนของแต่ละอันจริงๆ (5) บันทึกสิ่งที่รันจริง ไม่ใช่สิ่งที่ตั้งไว้

สิ่งที่พิสูจน์ด้วยการรันจริงและน่าจะเป็นประเด็นสำคัญสำหรับเพื่อน

- เกณฑ์คุณภาพแบบ adaptive ไม่เคยทำงานจริง ในระบบที่ deploy อยู่ ทุกรอบใช้ค่าฐาน 90.0 เพราะคอนเทนเนอร์ Spark ไม่มีตัวแปร ELASTICSEARCH_URL (BB-18)
- ฟอรัมกฎ Spark มี 8 ช่อง ใช้ตัดสินจริง 3 ช่อง (BB-20)
- เลือกคีย์ที่ไม่มีในไฟล์ หรืออัปโหลดไฟล์ที่ไม่มีคอสัมน์ semester แล้ว "แถวซ้ำ" เพิ่มจาก 30 เป็น 213 แถว โดยป้ายยังเขียนคีย์เดิม (BB-12, BB-13)
- บนหน้า Rule วงกลมตายตัว 95 / 99 / 99 ชัดกับตารางใต้การ์ดที่คำนวณจริง 93.7 / 97.1 / 98.1 (BB-08)
- ตัวคูณ k เปลี่ยนจาก 3.0 เป็น 1.5 แล้วจับแถวชุดเดียวกัน (20 แถว) บนไฟล์ทดสอบ ผู้ใช้จึงไม่เห็นค่า k ทำอะไร (BB-09)

คำถามที่ควรยืนยันกับเพื่อน

1. ผลงานรอบนี้คือรายงานข้อเสนอ หรือต้องลงมือแก้ระบบด้วย
2. เมื่อซ่อมเกณฑ์ adaptive ให้ทำงานจริง จะ จำกัดเกณฑ์ไว้ไม่ให้สูงกว่าค่าฐาน หรือรับพฤติกรรมเดิมของสูตร (gov_data จะได้เกณฑ์ 100.0 และตารางที่ต่ำคงที่จะหยุดถูกเตือน)
3. ไอคอน ? และแบนเนอร์ที่เคยเพิ่มไว้ (เป็นข้อความ) จะเก็บไว้หรือเอาออกเมื่อทำแบบใหม่แล้ว
4. จุดไหนที่เพื่อนเห็นว่าเป็น black box แต่ไม่อยู่ในรายการ 30 จุดนี้

ข้อมูลที่ใช้ทดสอบ

- **ฝั่งการ์ด:** sample_data/student_scores_sample.csv 1,030 แถว (นักเรียน 250 คน) ผลที่คาดด้วยกฎเริ่มต้น: Clean 915 / Review 20 / Quarantine 95 ตรงกับผลรันจริง ผลทั้ง 10 สถานการณ์อยู่ที่ evidence/interactive-scenarios.json
- **ฝั่ง Spark:** ผลรันจริงใน Elasticsearch ของระบบนี้ สามตาราง: gov_data (86 รอบ ใช้ดูเกณฑ์ adaptive), grocery_sales (7 รอบ มีแถวกักกันจริง), benchmark_test (28 รอบ คะแนนต่ำกว่าเกณฑ์)
- **หมายเหตุ:** ใน Elasticsearch มีเอกสารผลรันตัวอย่างจาก api/seed_es.py ปนอยู่ รายงานใช้เฉพาะเอกสารที่มีฟิลด์ effective_quality_threshold ซึ่งมีแต่เอนจิน Spark ที่เขียน
- ผลคำนวณด้วยมือทั้งหมดของฝั่ง Spark: evidence/spark-adaptive-threshold-worked.txt

วิธีรันหลักฐานซ้ำ

ทุกคำสั่งรันจากรากของ repo ใน Git Bash และไม่เปลี่ยนค่าในระบบ

ฝั่งการ์ด (รันบนสำเนาชั่วคราว ไม่แตะชุดข้อมูลและสถานะที่ใช้อยู่):

```
MSYS_NO_PATHCONV=1 docker compose cp sample_data/student_scores_sample.csv api:/tmp/tr_sample.csv
PYTHONIOENCODING=utf-8 MSYS_NO_PATHCONV=1 docker compose exec -T api python - < docs/transform-report/tools/capture_interactive.py > docs/transform-report/evidence/interactive-scenarios.json
```

ฝั่ง Spark (อ่านอย่างเดียว): ดึงจำนวนรอบต่อตาราง และเกณฑ์ที่บันทึกไว้ต่อตาราง

```
ES_PASS=$(MSYS_NO_PATHCONV=1 docker compose exec -T api printenv ELASTICSEARCH_PASSWORD | tr -d '\r')
curl -s -u "elastic:$ES_PASS" "http://localhost:9200/sdoqap_quality_runs/_search" -H "Content-Type: application/json" -d '{"size":0,"aggs":{"t":{"terms":{"field":"table_name.keyword","size":20},"thr":{"terms":{"field":"effective_quality_threshold","size":10}}}}}'
```

ตรวจการอ้างอิงโค้ดของรายงาน:

```
PYTHONIOENCODING=utf-8 python docs/whitebox-report/tools/check_citations.py docs/transform-report/*.md
```

รายละเอียดคำสั่งตั้งประวัติรอบของแต่ละตาราง และการรันฟังก์ชันคำนวณเกณฑ์ในคอนเทนเนอร์ `spark-master` อยู่ใน `evidence/spark-adaptive-threshold-probe.txt`

ส่วนที่ 1A: กฎ 3 การดบนหน้า Expectations & Alerts

เอกสารนี้ตอบคำถาม 4 ข้อของเพื่อนสำหรับ **การดกฎ 3 ในด้านบนของหน้า Rule** (ช่วงคะแนนและค่าว่าง / ห้ามคีย์ซ้ำ / ค่าผิดปกติ) ซึ่งเป็น "เอนจินแบบโต้ตอบ" ที่ทำงานในหน่วยความจำของ api ส่วนกฎรายการที่รันด้วย Spark อยู่ในเอกสาร 02-spark-table-rules.md

คำศัพท์ที่ใช้ในเอกสารนี้

- pandas / DataFrame:** โลกบรรทัดตารางของ Python DataFrame คือตารางข้อมูลที่ถูกโหลดไว้ในหน่วยความจำ
- mask:** รายการค่า จริง/เท็จ หนึ่งค่าต่อหนึ่งแถว นอกเหนือจากนั้น "โดนกฎ" หรือไม่ เหมือนปากกาเมจิกที่ไฮไลต์แถวที่ผิด
- Q1 / Q3 / IQR:** เรียงค่าจากน้อยไปมาก Q1 คือค่าที่ตำแหน่ง 25% Q3 คือค่าที่ตำแหน่ง 75% และ IQR = Q3 – Q1 บอกความกว้างของช่วง "ค่าปกติตรงกลาง"
- รั้ว Tukey (Tukey fence):** เส้นตัดสินค่าผิดปกติ = Q3 + k × IQR (รั้วบน) และ Q1 – k × IQR (รั้วล่าง) ค่าที่พ้นรั้วถือว่าผิดปกติ ตัวคูณ k ยิ่งมาก รั้วยิ่งห่าง ยิ่งตัดน้อย
- คีย์ประกอบ (composite key):** กลุ่มคอลัมน์ที่ใช้ตัดสินว่าสองแถว "เป็นเรื่องเดียวกัน" เช่น รหัสนักเรียน + วิชา + เทอม

ข้อมูลที่ใช้ยกตัวอย่าง: sample_data/student_scores_sample.csv (1,030 แถว มีปัญหาที่ฝังไว้ให้รู้จำนวนแน่นอน) ตัวเลขทุกตัวมาจากการรันเอนจินจริงด้วย docs/transform-report/tools/capture_interactive.py เก็บผลไว้ที่ docs/transform-report/evidence/interactive-scenarios.json โดยรันบนสำเนาชั่วคราว ไม่กระทบข้อมูลที่ใช้อยู่ในระบบ

ภาพรวมใน 5 บรรทัด

- การ์ด 3 ใบคือสวิตช์กับตัวเลขชุดเดียวกัน ส่งเข้าเอนจินตัวเดียวคือ _recompute_interactive_state (api/app/api/whitebox.py:1248)
- ทุกครั้งที่แก้ค่านบนการ์ด หน้าเว็บส่งค่าทั้งชุดไปทันที และเอนจินคำนวณใหม่ทั้งไฟล์ทันที ไม่มีสถานะ "ร่าง" ปุ่ม "บันทึกกฎ" แลส่งค่าเดิมซ้ำ (ข้อ 2)
- เอนจินตรวจสอบลำดับจริง: ซ้ำ → ว่าง/นอกช่วงคะแนน → ค่าผิดปกติ ซึ่งไม่ตรงกับเลขการ์ด 1→2→3 (ข้อ 3)
- ผลลัพธ์คือติดป้าย 3 คอลัมน์ให้ทุกแถว แล้วแยกเป็น 3 ไฟล์ Clean / Review / Quarantine เอนจินนี้ **ไม่แก้ค่าในข้อมูล** นอกจากค่าที่ผู้ตรวจพิมพ์แก้เองรายแถว (ข้อ 4)
- มีจุดที่ป้ายบนหน้าจอไม่ตรงกับสิ่งที่คำนวณจริงอย่างน้อย 3 จุด: คีย์ที่ไม่มีในไฟล์, ป้าย > 12.0h, และวงกลมเปอร์เซ็นต์คงที่ (สรุปไว้ท้ายเอกสาร)

ข้อ 1: ตั้งกฎไปทำไม แต่ละกฎกันปัญหาอะไร

การ์ด	ปัญหาที่กัน	ตัวอย่างแถวจริงจากไฟล์ทดสอบ	ถ้าปิดกฎนี้
1 ช่วงคะแนนและค่าว่าง	คะแนนที่เป็นไปไม่ได้ (ติดลบ, เกิน 100) หรือไม่มีคะแนนเลย ทำให้คะแนนเฉลี่ยและอันดับเพี้ยน	แถว #2 score ว่าง → Missing Score / แถว #21 score = -5.0 → Invalid Score Range	แถวเหล่านี้ไหลเข้า Clean ทั้งหมด (40 ว่าง + 25 นอกช่วง = 65 แถว)
2 ห้ามคีย์ซ้ำ	บันทึกเดียวกันเข้ามาสองครั้ง (นักเรียนคนเดียว วิชาเดียว เทอมเดียว) ทำให้นับซ้ำ	แถว #219 เป็นสำเนาของแถวก่อนหน้า → Duplicate	30 แถวสำเนาถูกนับเป็นข้อมูลจริง

การ์ด	ปัญหาที่กัน	ตัวอย่างแถวจริงจากไฟล์ทดสอบ	ถ้าปิดกฎนี้
3 ค่าผิดปกติ	ค่าที่ห่างจากส่วนใหญ่มาก อาจเป็นการกรอกผิด เช่น เรียน 41 ชั่วโมง ในขณะที่คนอื่น 1-10	แถว #27 study_hours = 41 → Study Hours Outlier	ปิดแล้ว (S9) Clean เพิ่มจาก 915 เป็น 935 และคะแนนคุณภาพเพิ่มจาก 88.8% เป็น 90.8% เพราะ 20 แถวผิดปกติถูกนับว่าดี

แถวตัวอย่างมาจาก S1_default.examples ส่วนตัวเลขกรณีปิดกฎมาจาก S9_rule3_off ในไฟล์หลักฐาน

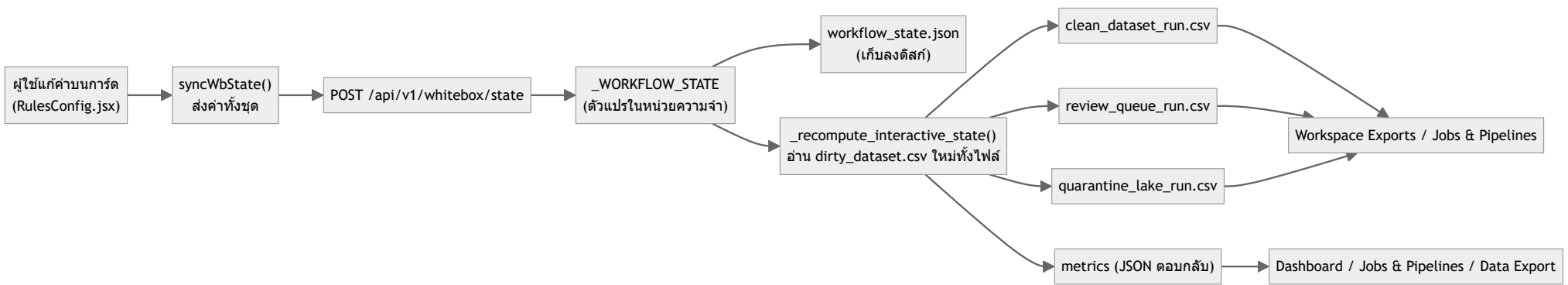
ข้อสังเกตเรื่องความแรงของแต่ละกฎ: โดยค่าเริ่มต้น กฎ 1 และ 2 ส่งแถวไป Quarantine (กักกัน) ส่วนกฎ 3 ส่งไป Review (รอคนตรวจ) เพราะ "ผิดปกติทางสถิติ" ไม่ได้แปลว่า "ผิด" เสมอไป

ความสามารถที่เอนจินมีแต่หน้าจอก็ไม่มีช่องให้ตั้ง

- null_policy = "adaptive_5": ให้ค่าว่างไป Review แทนการกักกัน ถ้าสัดส่วนค่าว่างไม่เกิน max_null_pct (api/app/api/whitebox.py:1295-1296 , :1324)
- dedup_strategy = "review_all": ให้แถวซ้ำไป Review แทนการกักกัน (api/app/api/whitebox.py:1319)

ทั้งสามค่านี้ (null_policy, max_null_pct, dedup_strategy) มีอยู่ในหน้า Rule เป็นตัวแปรสถานะเท่านั้น ไม่มี input ให้ผู้ใช้เปลี่ยน (ui/src/pages/RulesConfig.jsx:113-116) และทุกครั้งที่เกิดการ์ดอื่น หน้าเว็บส่งค่าเริ่มต้นของสามตัวนี้ทับไปด้วย (ui/src/pages/RulesConfig.jsx:135-138) ดังนั้นถ้าใครตั้งค่าเหล่านี้ผ่าน API มันจะถูกรีเซ็ตเมื่อมีคนแตะการ์ด

ข้อ 2: ตั้งค่าแล้ว ค่าถูกส่งไปไหน



ขั้นที่ 1: หน้าเว็บส่งค่าทั้งหมดทุกครั้งที่มีการเปลี่ยน

```
// ui/src/pages/RulesConfig.jsx:130-149
const syncWbState = async (overrides = {}) => {
  try {
    const payload = {
```

```

min_score: Number(overrides.min_score ?? (wbMinRange === "" ? 0 : wbMinRange)),
max_score: Number(overrides.max_score ?? (wbMaxRange === "" ? 100 : wbMaxRange)),
null_policy: overrides.null_policy ?? wbNullPolicy,
max_null_pct: Number(overrides.max_null_pct ?? (wbMaxNullPct === "" ? 5.0 : wbMaxNullPct)),
composite_key: overrides.composite_key ?? wbCompositeKey,
dedup_strategy: overrides.dedup_strategy ?? wbDedupStrategy,
tukey_multiplier: String(overrides.tukey_multiplier ?? wbTukeyMultiplier),
custom_upper_fence: Number(overrides.custom_upper_fence ?? (wbCustomUpperFence === "" ? 12.0 : wbCustomUpperFence)),
rule1_confirmed: overrides.rule1_confirmed ?? wbRule1Confirmed,
rule2_confirmed: overrides.rule2_confirmed ?? wbRule2Confirmed,
rule3_confirmed: overrides.rule3_confirmed ?? wbRule3Confirmed
};
const res = await fetch("/api/v1/whitebox/state", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload)
});

```

บรรทัด	ทำอะไร
133-134	ส่ง min_score, max_score จากช่อง input (ช่องว่างจะถูกแทนด้วย 0 / 100)
135-138	ส่ง null_policy, max_null_pct, composite_key, dedup_strategy จากตัวแปรสถานะ (ที่ไม่มี input ให้แก่)
139-140	ส่ง tukey_multiplier และ custom_upper_fence (ช่องว่างแทนด้วย 12.0)
141-143	ส่งสวิตช์เปิด/ปิดของกฎ 1, 2, 3
145-149	POST ไปที่ /api/v1/whitebox/state พร้อม JSON ทั้งหมด

อินพุตทุกช่องเรียก syncWbState ทันทีที่เปลี่ยน เช่น สวิตช์กฎ 1 (ui/src/pages/RulesConfig.jsx:645), ช่วงคะแนน (:664, :677), คีย์ (:723), ตัวคูณ k (:774) จึงไม่มีชั้น "ร่าง แล้วค่อยยืนยัน"

ปุ่ม "บันทึกกฎ" ทำอะไรจริง

```

// ui/src/pages/RulesConfig.jsx:191-200
const handleConfirmWhiteBoxRules = async () => {
  setWbConfirming(true);
  try {
    const updated = await syncWbState();
  }
}

```

```
setWbConfirmedAt(new Date().toLocaleTimeString());
const m = updated?.metrics || wbLiveMetrics;
setActionResult({
  success: true,
  message: `ยืนยันและประมวลผลกฎบน ${wbFmt(wbTotalRows)} แถวสำเร็จ: Clean ${wbFmt(m?.clean_rows)} แถว | Review ${wbFmt(m?.review_rows)} แถว | Quarantine ${wbFmt(m?.quarantine_rows)} แถว`
});
```

บรรทัด	ทำอะไร
194	เรียก syncWbState() โดยไม่ส่งค่าใหม่ ดังนั้นส่ง ค่าเดิมซ้ำ ทั้งชุด
195	บันทึกเวลาไว้แสดงบนปุ่มว่า "บันทึกแล้ว"
197-200	แสดงข้อความ "ยืนยันและประมวลผลกฎ ... สำเร็จ" พร้อมตัวเลขจากผลคำนวณ

syncWbState กลืนข้อผิดพลาดแล้วคืน null (ui/src/pages/RulesConfig.jsx:156-159) แต่ handleConfirmWhiteBoxRules ไม่ตรวจสอบว่าเป็น null และแสดงข้อความสำเร็จเสมอ (:196-200) ถ้า api ล่ม ข้อความก็ยังไม่บอกว่าสำเร็จ โดยตัวเลขที่แสดงเป็นค่าเก่าจากหน้าจอ

ขั้นที่ 2: ฝัง api เก็บค่าและคำนวณใหม่ทันที

```
# api/app/api/whitebox.py:1418-1444
@router.post("/state", dependencies=[Depends(require_session)])
def update_workflow_state(payload: Dict[str, Any] = Body(default_factory=dict)):
    for k, v in payload.items():
        if k in _WORKFLOW_STATE:
            if k == "selected_findings":
                current_norm = _normalize_selected_findings(_WORKFLOW_STATE.get("selected_findings"))
                if isinstance(v, dict):
                    current_norm.update({k2: bool(v2) for k2, v2 in v.items() if k2 in ("range", "duplicate", "outlier")})
                    _WORKFLOW_STATE["selected_findings"] = current_norm
            else:
                _WORKFLOW_STATE["selected_findings"] = _normalize_selected_findings(v)
        elif k == "row_decisions" and isinstance(v, dict):
            if len(v) == 0:
                _WORKFLOW_STATE["row_decisions"] = {}
            else:
                _WORKFLOW_STATE["row_decisions"].update(v)
        elif k == "row_edits" and isinstance(v, dict):
```

```
        if len(v) == 0:
            _WORKFLOW_STATE["row_edits"] = {}
        else:
            _WORKFLOW_STATE["row_edits"].update(v)
    else:
        _WORKFLOW_STATE[k] = v
    if payload.get("confirm_rules"):
        _WORKFLOW_STATE["confirmed_at"] = datetime.now(timezone.utc).strftime("%H:%M:%S")
    _save_workflow_state()
    return _recompute_interactive_state()
```

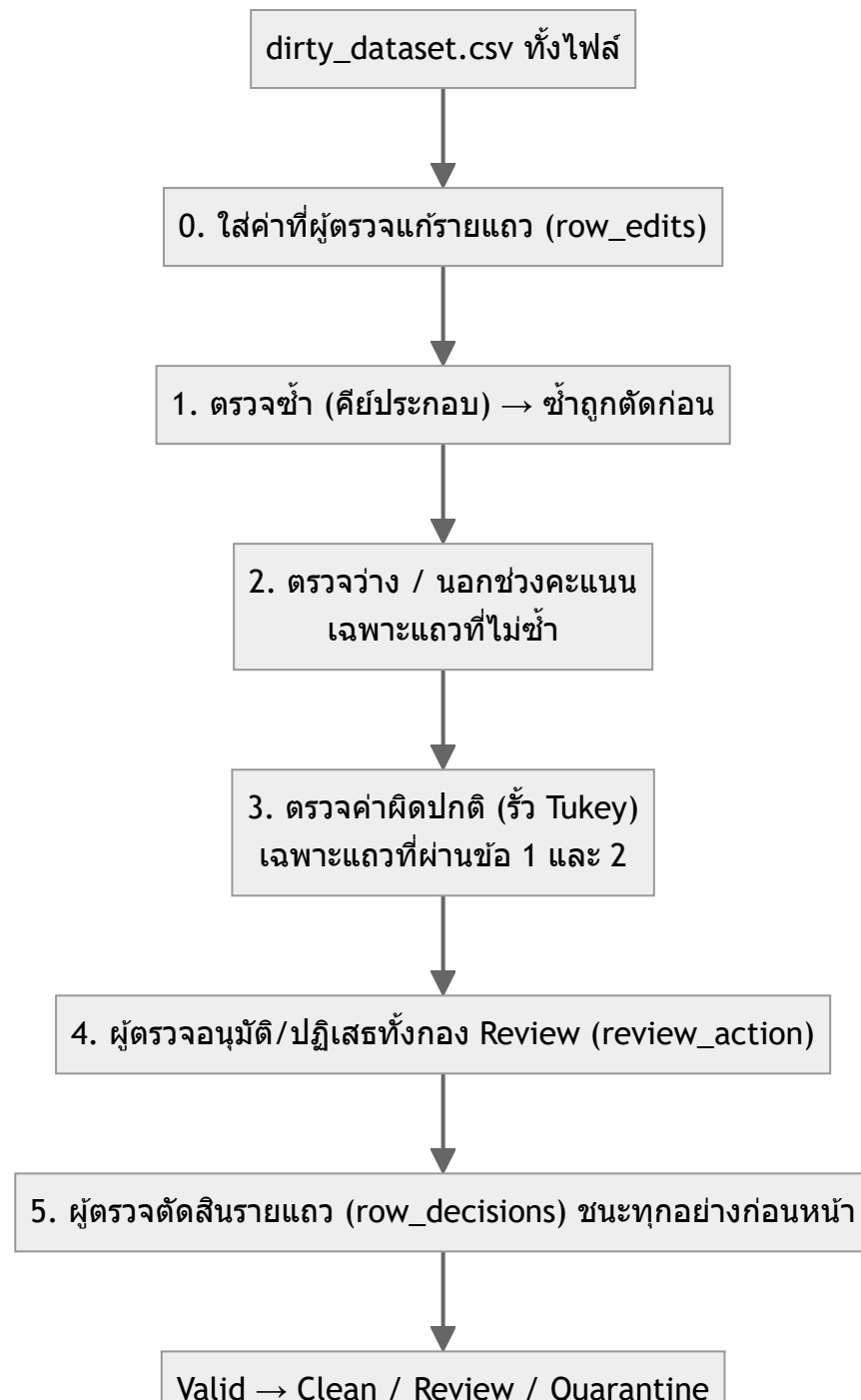
บรรทัด	ทำอะไร
1420-1421	วนทุกคีย์ที่ส่งมา และรับเฉพาะคีย์ที่มีอยู่แล้วใน <code>_WORKFLOW_STATE</code> คีย์แปลกปลอมถูกทิ้งเจียนๆ
1422-1428	<code>selected_findings</code> (สวิตช์ "ตรวจเรื่องนี้ใหม่" ที่ตั้งจากหน้า Data Ingestion) ถูก normalize แล้วรวมเข้าไป
1429-1438	<code>row_decisions</code> และ <code>row_edits</code> (การตัดสินใจรายแถวจากหน้า Pipeline) ถูกรวมเข้าไป ส่งก่อนวางคือล่างทั้งหมด
1439-1440	ค่าอื่นทั้งหมดถูกเขียนทับตรงๆ ไม่ตรวจสอบค่า เช่น <code>min_score</code> มากกว่า <code>max_score</code> ก็รับ
1443	<code>_save_workflow_state()</code> เขียน <code>workflow_state.json</code> ลงดิสก์ (<code>api/app/api/whitebox.py:1201-1208</code>) เพื่อให้การถอดการรีสตาร์ท
1444	คำนวณใหม่ทันทีแล้วตอบกลับ

การ `GET /api/v1/whitebox/state` ก็เรียกเอนจินคำนวณใหม่ทั้งไฟล์เช่นกัน (`api/app/api/whitebox.py:1413-1415`) ทุกหน้าที่โหลดสถานะจึงเห็นผลตามค่าปัจจุบันเสมอ

กฎแต่ละข้อมีสวิตช์สองชั้นอยู่คนละหน้า กฎจะทำงานเมื่อสวิตช์บนการ์ด (`rule1_confirmed` ฯลฯ) เปิด และ `selected_findings` ของหน้า Data Ingestion เปิดด้วย (`api/app/api/whitebox.py:1286-1289`) ตัวหลังตั้งจากช่องติ๊กในหน้า Data Ingestion (`ui/src/pages/Ingestion.jsx:241-249`) ผู้ใช้ที่ปิดกฎบนการ์ดจะเห็นผลชัด แต่ผู้ใช้ที่ปิดจากหน้า Ingestion แล้วเห็นการ์ดยังเปิดอยู่ จะไม่รู้ว่ากฎไม่ได้ทำงาน

ข้อ 3: ตัวเลขในกฎถูกเอาไปคำนวณยังงี้

เอนจินทำงานเป็นลำดับดังนี้ (ลำดับจริงตามโค้ด):



3.1 ตรวจซ้ำ (ทำก่อนสุด)

```
# api/app/api/whitebox.py:1278-1284
comp_key_str = _WORKFLOW_STATE.get("composite_key", "student_id + course + semester")
comp_cols = [c.strip() for c in comp_key_str.split("+") if c.strip() in df.columns]
if not comp_cols:
    comp_cols = ["student_id", "course", "semester"]
dedup_strategy = _WORKFLOW_STATE.get("dedup_strategy", "keep_first_quarantine")
tukey_str = str(_WORKFLOW_STATE.get("tukey_multiplier", "3.0"))
tukey_mult = float(tukey_str) if tukey_str in ("3.0", "1.5") else 3.0
```

บรรทัด	ทำอะไร
1278	อ่านสตริงคีย์ที่ผู้ใช้เลือก เช่น student_id + course + semester
1279	แยกด้วย + แล้วเก็บ เฉพาะชื่อที่มีเป็นคอลัมน์จริงในไฟล์
1280-1281	ถ้าไม่เหลือคอลัมน์เลย กลับไปใช้คีย์เริ่มต้น student_id, course, semester โดยไม่แจ้งใคร
1283-1284	ตัวคูณ k ที่ไม่ใช่ 3.0 หรือ 1.5 (รวมถึง custom) ถูกแปลงเป็น 3.0 ในสูตร (แต่ดู 3.3 เรื่อง custom)

```
# api/app/api/whitebox.py:1291-1298
# 1. Duplicate mask (Gate 2)
dup_mask = df.duplicated(subset=comp_cols, keep="first") if r2_active else pd.Series(False, index=df.index)

# 2. Missing score & out-of-range masks (Gate 1, evaluated on non-duplicate rows)
observed_null_pct = (df["score"].isna().sum() / total_rows) * 100.0 if total_rows > 0 else 0.0
should_quarantine_nulls = (null_policy == "strict_0") or (observed_null_pct > max_null_pct)
null_mask = (~dup_mask) & df["score"].isna() if r1_active else pd.Series(False, index=df.index)
range_mask = (~dup_mask) & (~df["score"].isna()) & ((df["score"] < min_s) | (df["score"] > max_s)) if r1_active else pd.Series(False, index=df.index)
```

บรรทัด	ทำอะไร
1292	df.duplicated(subset=comp_cols, keep="first") แถวแรกของแต่ละกลุ่มคีย์ผ่าน แถวถัดๆ ไปคือ "ซ้ำ" (ถ้ากฎ 2 ปีด ไม่มีแถวซ้ำเลย)
1295	สัดส่วนค่าว่างที่สังเกตได้ = จำนวนแถว score ว่าง ÷ แถวทั้งหมดทุกแถว × 100
1296	ค่าว่างจะถูก "กักกัน" ถ้านโยบายเป็น strict_0 หรือสัดส่วนที่สังเกตได้มากกว่าเพดาน

บรรทัด	ทำอะไร
1297	แถว "ว่าง" คือ ไม่ซ้ำ และ score ว่าง (ถ้ากฎ 1 เปิด)
1298	แถว "นอกช่วง" คือ ไม่ซ้ำ, score ไม่ว่าง, และต่ำกว่า min หรือสูงกว่า max

ตัวอย่าง: ส่งแถวเข้าไป Review (S6): เมื่อ dedup_strategy = "review_all" แถวซ้ำ 30 แถวไปที่ Review แทน Quarantine (api/app/api/whitebox.py:1319) ผลที่วัดได้: Review เพิ่มขึ้น 20 เป็น 50, Quarantine ลดจาก 95 เป็น 65, Clean เท่าเดิม 915 และ gate2_quarantined เป็น 0 (api/app/api/whitebox.py:1379)

ตัวอย่าง: คีย์ที่ไม่มีในไฟล์ (S7) หน้า Rule มีตัวเลือกคือ record_id (ui/src/pages/RulesConfig.jsx:728) แต่ไฟล์ทดสอบไม่มีคอลัมน์ record_id เมื่อเลือก ผลลัพธ์ตรงกับ S1 ทุกตัวเลข (Clean 915 / Review 20 / Quarantine 95 ซ้ำ 30 แถว) เพราะเอนจินใช้คีย์เริ่มต้นแทน แต่ป้ายเหตุผลที่ติดบนแถวเขียนว่า Composite Key Uniqueness (record_id) ตามที่ผู้ใช้เลือก (api/app/api/whitebox.py:1322) ป้ายจึงบอกคีย์ที่ **ไม่ได้ถูกใช้จริง** (ดู S7_unknown_key_record_id.rule_applied_counts)

3.2 ตรวจว่างและนอกช่วงคะแนน

ตรวจบนแถวที่ไม่ซ้ำเท่านั้น (บรรทัด 1297-1298 ข้างบน) ป้ายที่ติดคือ Value Range Check [min, max] (api/app/api/whitebox.py:1331)

ตัวอย่างช่วงคะแนน (S3): เปลี่ยน min_score จาก 0 เป็น 40 แถวนอกช่วงเพิ่มจาก 25 เป็น 29 (คะแนนที่อยู่ในช่วง 0 ถึงต่ำกว่า 40 มี 4 แถวในไฟล์ทดสอบ ตรวจนับจากไฟล์โดยตรง) Clean ลดจาก 915 เหลือ 911 และคะแนนคุณภาพลดจาก 88.8% เป็น 88.4%

ตัวอย่างนโยบายค่าว่าง (S4 เทียบ S5): ไฟล์ทดสอบมี score ว่าง 40 แถวจาก 1,030

คำนวณด้วยมือจากสูตรที่ api/app/api/whitebox.py:1295
สัดส่วนค่าว่างที่สังเกตได้ = 40 / 1030 * 100 = 3.883%

สถานการณ์	เงื่อนไขที่ บรรทัด 1296	ค่าว่างไปที่	ผลที่วัดได้
S4: adaptive_5, เพดาน 5%	3.883 > 5 เป็นเท็จ	Review	Review 60 (เดิม 20 + ว่าง 40) / Quarantine 55
S5: adaptive_5, เพดาน 3%	3.883 > 3 เป็นจริง	Quarantine	เหมือน S1: Review 20 / Quarantine 95

ข้อสังเกต: ตัวเลข "สัดส่วนที่สังเกตได้" ในสูตรนี้หารด้วยแถวทั้งหมด **รวมแถวซ้ำ** ทั้งที่แถวซ้ำถูกตัดออกก่อนแล้ว จึงไม่ใช่สัดส่วนของแถวที่เหลือ

3.3 ตรวจค่าผิดปกติ (รั้ว Tukey)

```
# api/app/api/whitebox.py:1300-1313
# 3. Tukey IQR Outlier mask (Gate 3, evaluated on rows passing Gate 1 & Gate 2)
clean_hours = df["study_hours"].dropna()
q1 = float(clean_hours.quantile(0.25))
```

```
q3 = float(clean_hours.quantile(0.75))
iqr = float(q3 - q1)
if _WORKFLOW_STATE.get("custom_upper_fence") is not None and tukey_str == "custom":
    upper_fence = float(_WORKFLOW_STATE["custom_upper_fence"])
else:
    upper_fence = float(q3 + tukey_mult * iqr)
    _WORKFLOW_STATE["custom_upper_fence"] = upper_fence
lower_fence = float(q1 - tukey_mult * iqr)

passed_g1_g2 = (~dup_mask) & (~null_mask) & (~range_mask)
outlier_mask = passed_g1_g2 & (~df["study_hours"].isna()) & ((df["study_hours"] > upper_fence) | (df["study_hours"] < lower_fence)) if r3_active else pd.Series(False, index=df.index)
```

บรรทัด	ทำอะไร
1301	เอาคอลัมน์ <code>study_hours</code> ทุกแถว (ตัดค่าว่างออก) มาคำนวณ ไม่ได้กรองแถวซ้ำหรือแถวที่กักกันแล้วออกก่อน
1302-1304	หา Q1, Q3 และ $IQR = Q3 - Q1$
1305-1306	ถ้าเลือกโหมด <code>custom</code> ใช้ค่ารับนที่ผู้ใช้พิมพ์ตรงๆ
1307-1309	มีจะนั้นรับน = $Q3 + k \times IQR$ และ เขียนค่ารับนนี้ทับ <code>custom_upper_fence</code> ในสถานะ
1310	รัวล่าง = $Q1 - k \times IQR$ (คำนวณเสมอ)
1312	แถวที่ "ผ่านด่านก่อนหน้า" คือไม่ซ้ำ ไม่ว่าง ไม่นอกช่วง
1313	ค่าผิดปกติ = ผ่านด่านก่อนหน้า และ <code>study_hours</code> ไม่ว่าง และ (เกินรับน หรือ ต่ำกว่ารัวล่าง) ถ้ากฎ 3 เปิด

ตัวอย่าง (S1 เทียบ S2): จากหลักฐาน $Q1 = 3.0, Q3 = 8.0$

```
คำนวณด้วยมือจากสูตรที่ api/app/api/whitebox.py:1304, 1308, 1310
IQR          = Q3 - Q1 = 8.0 - 3.0 = 5.0
รับน k=3.0   = Q3 + 3.0 * IQR = 8.0 + 15.0 = 23.0    (ตรงกับ upper_fence ใน S1)
รัวล่าง k=3.0 = Q1 - 3.0 * IQR = 3.0 - 15.0 = -12.0
รับน k=1.5   = 8.0 + 1.5 * 5.0 = 15.5              (ตรงกับ upper_fence ใน S2)
รัวล่าง k=1.5 = 3.0 - 1.5 * 5.0 = -4.5
```

รัวล่างเป็นค่าติดลบทั้งสองกรณี และ `study_hours` ในไฟล์ไม่เคยติดลบ จึงไม่จับแถวใดเลย

ผลที่วัดได้: S1 และ S2 จับ **20 แกวเท่ากัน** เพราะ 20 แกวที่ฝั่งไว้มี `study_hours` 35–60 ซึ่งเกินทั้ง 23.0 และ 15.5 (ตรวจนับจากไฟล์: แกวที่ `study_hours > 15.5` มี 20 แกว และ `> 12` ก็ 20 แกว) การเลื่อน `k` บนไฟล์นี้จึง **ไม่เปลี่ยนผลลัพธ์แม้แต่แกวเดียว** ผู้ใช้จึงไม่มีทางเห็นจากผลว่า `k` ทำหน้าที่อะไร

ตัวอย่างป้าย > 12.0h (ตรงข้ามความจริง) ตัวเลือกใน dropdown เขียนว่า `3.0× IQR (> 12.0h)` และ `1.5× IQR (> 9.0h)` (`ui/src/pages/RulesConfig.jsx:778-779`) เป็นข้อความตายตัวที่เขียนไว้ในโค้ดโดยสมมติชุดข้อมูลเดิม และเมื่อเปลี่ยนตัวเลือก หน้าเว็บส่งร้ว `12.0 / 9.0` ไปด้วย (`:772-774`) แต่เอนจินไม่ใช้ค่านั้น เพราะใช้เฉพาะเมื่อโหมดเป็น `custom` (`api/app/api/whitebox.py:1305`) และ dropdown ไม่มีตัวเลือก `custom` ให้เลือก บนไฟล์ทดสอบร้วจริงคือ **23.0 ชั่วโมง** ไม่ใช่ 12.0

ตัวอย่างโหมด custom (S8): เมื่อส่ง `custom` กับ 12.0 ผ่าน API ป้ายเปลี่ยนเป็น `Tukey IQR Fence (> 12.0h)` แต่ผลจับ 20 แกวเท่าเดิม เพราะแกวผิดปกติทั้ง 20 แกวเกิน 12 อยู่แล้ว

ตัวอย่างปิดกฎ 3 (S9): ไม่มีแกวใดถูกส่ง Review (Review = 0) Clean = 935

3.4 การตัดสินใจโดยมนุษย์ (ทับผลของกฎ)

```
# api/app/api/whitebox.py:1337-1360
# Apply Human-in-the-Loop Review Action ("KEEP" | "APPROVE" | "REJECT")
rev_action = _WORKFLOW_STATE.get("review_action", "KEEP")
if rev_action == "APPROVE":
    df.loc[df["whitebox_status"] == "Review", "whitebox_rule_applied"] = "Human Approved -> Clean Asset"
    df.loc[df["whitebox_status"] == "Review", "whitebox_status"] = "Valid"
elif rev_action == "REJECT":
    df.loc[df["whitebox_status"] == "Review", "whitebox_rule_applied"] = "Human Rejected -> Quarantine Lake"
    df.loc[df["whitebox_status"] == "Review", "whitebox_status"] = "Quarantine"

# Apply individual row overrides if any
row_decisions = _WORKFLOW_STATE.get("row_decisions", {}) or {}
for row_key, decision in row_decisions.items():
    try:
        rid = int(str(row_key).replace("#", ""))
        if "dirty_row_id" in df.columns:
            mask = df["dirty_row_id"] == rid
            if decision == "APPROVE":
                df.loc[mask, "whitebox_status"] = "Valid"
                df.loc[mask, "whitebox_rule_applied"] = f"Row #{rid} Corrected & Approved"
            elif decision == "REJECT":
                df.loc[mask, "whitebox_status"] = "Quarantine"
                df.loc[mask, "whitebox_rule_applied"] = f"Row #{rid} Quarantined by Reviewer"
```

```
except Exception:
    pass
```

บรรทัด	ทำอะไร
1338-1341	ถ้า review_action = APPROVE แถวที่อยู่ในสถานะ Review ทั้งหมดกลายเป็น Valid
1342-1344	ถ้า REJECT แถว Review ทั้งหมดกลายเป็น Quarantine
1347-1358	row_decisions รายแถวเขียนทับสถานะและป้ายเหตุผล ไม่ว่าแถวนั้นจะติดกฎไหน หรือไม่ติดเลยก็ตาม
1359-1360	ถ้าเลขแถวอ่านไม่ออก ข้ามไปเจียบบๆ

ลำดับความสำคัญจากต่ำไปสูง: ผลของกฎ → review_action (เฉพาะแถว Review) → row_decisions (ทุกแถว) การแก้ค่ารายแถว row_edits ทำตั้งแต่ก่อนตรวจกฎ (api/app/api/whitebox.py:1260-1272) จึงเป็นค่าเดียวที่ทำให้ข้อมูลเปลี่ยนจริง เมื่อแถวถูกแก้แล้ว ป้ายที่ติดบนแถวไม่บอกค่าเดิม

ข้อ 4: หลังตรวจแล้ว ข้อมูลไปต่อที่ไหน

เมื่อทุกกฎและการตัดสินใจของผู้ตรวจจบ แต่ละแถวมีสถานะเดียว และเอนจินเขียนเป็น 3 ไฟล์ทุกครั้งที่คำนวณ (api/app/api/whitebox.py:1362-1372)

สถานะ	ไฟล์	หน้าที่อ่านต่อ
Valid (Clean)	clean_dataset_run.csv	Workspace Exports ดาวน์โหลด (api/app/api/whitebox.py:1447-1463) และ Jobs & Pipelines ดูตัวอย่าง (:1466-1496)
Review	review_queue_run.csv	ผู้ตรวจอนุมัติ/ปฏิเสธทั้งกอง (ui/src/pages/Pipeline.jsx:143, :337-351) หรือรายแถว (:106, :497-505) ตามข้อ 3.4
Quarantine	quarantine_lake_run.csv	ดาวน์โหลดเป็นบันทึกกักกัน และดูเหตุผลรายแถวได้

หน้าที่ใช้ผลนี้: Dashboard อ่านสถานะทุก 10 วินาที (ui/src/pages/Dashboard.jsx:100), Jobs & Pipelines อ่านสถานะและตารางรายแถว (ui/src/pages/Pipeline.jsx:171, :191), Workspace Exports อ่านตัวอย่างและดาวน์โหลด (ui/src/pages/DataExport.jsx:32, :90)

คะแนนคุณภาพ คำนวณที่ api/app/api/whitebox.py:1386 เป็น Clean ÷ ทั้งหมด

```
คำนวณด้วยมือจากสูตรที่ api/app/api/whitebox.py:1386
quality_score_pct = round(915 / 1030 * 100, 1) = round(88.83, 1) = 88.8    (ตรงกับ S1)
```

สิ่งที่ยืนยันได้: เอนจินนี้แค่ จัดประเภทและติดป้าย ไม่มีขั้นแก้ค่าอัตโนมัติ (ไม่เติมค่าว่าง ไม่ตัดค่าผิดปกติ ไม่รวมแถวซ้ำ) แถว Clean คือแถวที่ไม่โดนกฎใดและไม่ถูกผู้ตรวจปฏิเสธ ต่างจากฝั่ง Spark ที่ตัดแถวซ้ำและเขียน Delta จริง (ดู 02-spark-table-rules.md)

ข้อจำกัดที่ควรรู้: ไฟล์ทั้งสามถูกเขียนทับทุกครั้งที่มีการ GET หรือ POST สถานะ และตัวเลขทั้งหมดอยู่ในหน่วยความจำ + ไฟล์ของ api ไม่ผ่าน Elasticsearch หรือ HDFS จึงไม่มีประวัติว่ารอบก่อนใช้กฎทำอะไร

ข้อที่ต่างจากที่คาด

รายการนี้บันทึกสิ่งที่ตรวจโค้ดแล้วต่างจากข้อสันนิษฐานในแผนงาน หรือเป็นของใหม่ที่เจอระหว่างเขียน

1. ป้ายคีย์ไม่ตรงกับคีย์ที่ใช้จริง: เมื่อเลือกคีย์ที่ไม่มีในไฟล์ เอนจินใช้คีย์เริ่มต้นแต่ป้ายเขียนชื่อคีย์ที่เลือก (S7) เป็นความคลาดเคลื่อนที่ยืนยันด้วยการรัน ไม่ใช่แค่การอ่านโค้ด
2. ปุ่ม "บันทึกกฎ" แสดงสำเร็จแม้ api ล้มเหลว (ui/src/pages/RulesConfig.jsx:156-159, :194-200)
3. การเลื่อน k ไม่เปลี่ยนผลบนไฟล์ทดสอบ ทั้งที่มีจริงในโค้ด: S1 และ S2 จับ 20 แถวเท่ากัน เพราะค่าผิดปกติที่ฝังไว้ห่างจากรั้วทั้งสองมาก
4. สวิตช์สองชั้นต่อเนื่องกฎ อยู่คนละหน้า (การรัดกับ Data Ingestion)
5. ร้วล่างมีอยู่จริงแต่ไม่แสดงบนการ์ด ป้ายการ์ด 3 เขียนเฉพาะรั้วบน (ui/src/pages/RulesConfig.jsx:740)
6. Q1/Q3 คัดจากทุกแถวรวมแถวซ้ำและแถวที่กักกันแล้ว (api/app/api/whitebox.py:1301) ไม่ใช่เฉพาะแถวที่ผ่านด่านก่อนหน้านี้แบบที่ความคิดเห็นในโค้ด "evaluated on rows passing Gate 1 & Gate 2" ขวนให้เข้าใจ (ความคิดเห็นอยู่ที่ api/app/api/whitebox.py:1300) ร้วจึงอาจต่างจากที่คาดเมื่อข้อมูลมีแถวซ้ำมาก
7. ป้ายในโค้ดเรียกด่านตรวจซ้ำว่า "Gate 2" แต่รันก่อน "Gate 1" (api/app/api/whitebox.py:1291, :1294) และหน้า Jobs & Pipelines ใช้ชื่อ Gate 1 / Gate 2 ตามเลขนี้ (ui/src/pages/Pipeline.jsx:279-280) ซึ่งไม่ตรงกับลำดับที่ทำงานจริง

ส่วนที่ 1B: กฎรายการ (Spark) ในตั้งค่าขั้นสูง

เอกสารนี้ตอบคำถาม 4 ข้อของเพื่อนสำหรับ กฎรายการ ที่แก้ไขได้ในหน้า Expectations & Alerts (แท็บตาราง / ตั้งค่าขั้นสูง) กฎชุดนี้ไม่ได้ทำงานในหน้าเว็บ แต่ถูกบันทึกลงไฟล์ตั้งค่าแล้ว ถูกอ่านโดย **Spark** ตอนรอบตรวจครั้งถัดไป ส่วนการ์ด 3 ในอยู่ในเอกสาร 01-interactive-rules.md

คำศัพท์ที่ใช้ในเอกสารนี้

- Spark:** โปรแกรมประมวลผลข้อมูลก้อนใหญ่ที่รันแยกจากเว็บ (ในคอนเทนเนอร์ spark-master) อ่านไฟล์ข้อมูลจริงจาก HDFS ทั้งหมด
- Delta Lake / Delta table:** รูปแบบไฟล์ตารางบน HDFS ที่รองรับการอัปเดตแบบ "ถ้าดีก็ตรงให้แก้ ถ้าไม่ดีให้เพิ่ม" (MERGE) เหมือนตารางฐานข้อมูลที่เป็นไฟล์
- Elasticsearch (ES):** ฐานข้อมูลค้นหาที่ระบบใช้เก็บกฎ (sdoqap_rules_registry) และผลการรันทุกรอบ (sdoqap_quality_runs)
- Adaptive (ปรับตัวเอง):** ค่าเกณฑ์ที่ระบบคำนวณใหม่จากประวัติ แทนที่จะเป็นเลขคงที่
- IQR / Q1 / Q3 / ร้อย:** ความหมายเดียวกับเอกสาร 01 (ร้อยละ = $Q3 + k \times IQR$, ร้อยล่าง = $Q1 - k \times IQR$)
- Z-score:** จำนวน "ส่วนเบี่ยงเบนมาตรฐาน" ที่ค่าหนึ่งห่างจากค่าเฉลี่ยของคอลัมน์ ยิ่งมากยิ่งผิดปกติ
- Quarantine:** โชนกักกันแถวที่ไม่ผ่านกฎ เก็บเป็นตาราง Delta แยกตามรหัสรอบรัน (run_id)

ข้อมูลที่ใช้ยกตัวอย่าง: ผลการรันจริงที่เก็บอยู่ใน Elasticsearch ของระบบนี้ เก็บสำเนาไว้ใน docs/transform-report/evidence/

ตาราง	ใช้ทำอะไรในเอกสารนี้	ไฟล์หลักฐาน
gov_data (86 รอบ ประวัติมากที่สุด)	ตัวอย่างเกณฑ์ adaptive (ครบหน้าต่าง 15 รอบ)	spark-quality-runs-gov_data.json, spark-rules-gov_data.json
grocery_sales (7 รอบ)	ตัวอย่างแถวที่ถูกกักกันจริงและร้อย IQR จริง	spark-quality-runs-grocery_sales.json
benchmark_test (28 รอบ)	ตัวอย่างตารางที่คะแนนต่ำกว่าเกณฑ์	spark-quality-runs-benchmark_test.json

เหตุผลที่ไม่ใช้ตารางเดียว: gov_data ทุกรอบได้ 100% ไม่มีแถวถูกกักกันเลย จึงไม่มีเหตุการณ์การกักกันให้ดู ต้องใช้ grocery_sales เสริม ผลการคำนวณด้วยมือทั้งหมดอยู่ใน docs/transform-report/evidence/spark-adaptive-threshold-worked.txt

ภาพรวมใน 5 บรรทัด

- ค่าที่ตั้งบนฟอร์มจะถูกบันทึกลง rules_config.json และ Elasticsearch ทันทีที่กดบันทึก แต่ มีผลกับข้อมูลเมื่อ Spark รันรอบถัดไปเท่านั้น ไม่มีการคำนวณใหม่กับข้อมูลที่ผ่านมาแล้ว
- Spark ตรวจสอบลำดับ: ลบแถวซ้ำ → คีย์หลักว่าง → ค่าว่างในทุกคอลัมน์ → ชนิดข้อมูล → วันที่ว่าง → ค่าผิดปกติแบบ IQR → ค่าผิดปกติแบบ Z-score ทุกแถวที่ไม่ผ่านไปโชนกักกัน ไม่มีโชน Review แบบฝั่งการ์ด
- แถวที่ผ่านถูก MERGE เข้าตาราง active ที่สะสมข้อมูลต่อเนื่อง แล้วเทียบคะแนนกับเกณฑ์เพื่อตัดสินว่าแจ้งเตือนและเรียก Gold layer หรือไม่

4. จากฟอร์ม 8 ช่อง มี 5 ช่อง (ใน 4 หัวข้อ) ที่ไม่มีผลต่อการตัดสินใจ (ความอดทนต่อค่าว่าง, โหมดและค่าความหน่วงข้อมูล, ชื่อโมเดล AI, และโหมด adaptive ของเกณฑ์คุณภาพ) รายละเอียดที่ตาราง "ตั้งค่าไหนมีผลจริง"
5. เกณฑ์คุณภาพแบบ adaptive ไม่เคยทำงานจริงในระบบที่ deploy นี้ ทุกรอบที่บันทึกไว้ใช้ค่าฐาน (90.0) เพราะตัวแปรแวดล้อมที่ฟังก์ชันต้องใช้ไม่ถูกตั้งในคอนเทนเนอร์ Spark (พิสูจน์ที่ข้อ 3)

ข้อ 1: ตั้งกฎไปทำไม แต่ละกฎกันปัญหาอะไร

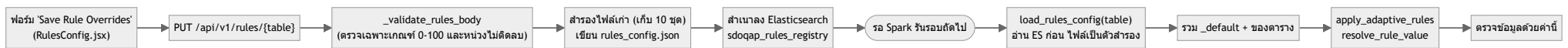
ตารางนี้ลำดับตามที่ Spark ตรวจจริง ป้ายในคอลัมน์ "เหตุผลที่บันทึก" คือข้อความใน reject_reason ของแถวที่ถูกกักกัน

ลำดับ	การตรวจ	กันปัญหาอะไร	เหตุผลที่บันทึกในแถวกักกัน	หลักฐาน
1	ลบแถวซ้ำ (auto-clean)	ข้อมูลเดียวกันเข้ามาสองครั้ง ทำให้นับซ้ำ	ไม่บันทึก: แถวซ้ำถูกลบทิ้ง ไม่ถูกกักกัน บันทึกแค่ข้อความ resolved_N_duplicates ในผลการรัน	spark/spark_quality_engine.py:2043-2060
2	คีย์หลักว่าง	แถวที่ระบุตัวตนไม่ได้ ไม่มีทางรวม (MERGE) เข้าตารางหลักได้	missing_primary_key	spark/spark_quality_engine.py:2062-2072
3	ค่าว่างในคอลัมน์อื่นทุกคอลัมน์	ข้อมูลไม่ครบ	null_value_in_<ชื่อคอลัมน์> (ตัวอย่างจริง: null_value_in ยอดขายรวม 4 แถวใน grocery_sales)	spark/spark_quality_engine.py:2074-2088
4	ชนิดข้อมูลตัวเลข	ตัวอักษรปนในคอลัมน์ตัวเลข	invalid_type_<ชื่อคอลัมน์>	spark/spark_quality_engine.py:2090-2110
5	วันที่ว่าง	แถวที่บอกไม่ได้ว่าเกิดเมื่อไร	missing_date	spark/spark_quality_engine.py:2112-2121
6	แถวซ้ำที่เหลือหลังตรวจ (dedup รอบสอง)	กันซ้ำอีกชั้นถ้า auto-clean ปิด	duplicate_records	spark/spark_quality_engine.py:2127-2137, :2227-2229
7	ค่าผิดปกติแบบ IQR	ค่าที่ห่างจากส่วนใหญ่ของคอลัมน์ตัวเลข	<คอลัมน์>=<ค่าจริง> (expected [ต่ำสุด, สูงสุด])	spark/spark_quality_engine.py:2231-2264
8	ค่าผิดปกติแบบ Z-score (ตายตัว 3.0)	ค่าที่ห่างเกิน 3 เท่าของส่วนเบี่ยงเบนมาตรฐาน	<คอลัมน์>_zscore=<ค่า> (val=... deviates > 3.0σ)	spark/spark_quality_engine.py:2266-2285
9	กฎที่เรียนรู้จาก Decision Tree (ถ้ามี induced ในตั้งค่า)	รูปแบบผิดปกติที่ระบบสรุปจากแถวที่กักกัน	induced_tree_rule_match	spark/spark_quality_engine.py:2292-2326

ข้อสังเกตที่แตกต่างจากฝั่งการ์ด: ข้อ 7 ให้ ค่าจริงและช่วงที่คาดไว้ในข้อความเหตุผล (เช่น ราคา=999 (expected [-12.5, 87.5])) ซึ่งเป็นรูปแบบที่ตามรอยได้ดีกว่าป้าย Value Range Check [0.0, 100.0] ของฝั่งการ์ด ดูโค้ดสร้างข้อความที่ spark/dynamic_rules_engine.py:550-559

สิ่งที่ไม่ได้อยู่ในตารางนี้แต่มีผลต่อการตัดสินใจ: เกณฑ์คะแนนคุณภาพ (quality_score_threshold) ไม่ได้กักกันแถวใด แต่ตัดสินใจรอบนี้ต้องแจ้งเตือนและต้องเรียกกระบวนการต่อไปหรือไม่ (ข้อ 4)

ข้อ 2: ตั้งค่าแล้ว ค่าถูกส่งไปไหน



ขั้นที่ 1: ฟอรั่มส่งกฎทั้งก่อนของตาราง

```
// ui/src/pages/RulesConfig.jsx:461-467
const res = await fetch(`/api/v1/rules/${tbl}`);
if (res.ok) {
  const data = await res.json();
  setTableRules(data.effective_rules);
  // Deep-clone snapshot of the as-fetched rules, kept untouched by edits, so the
  // save confirmation can show a real before/after diff instead of a generic message.
  setOriginalTableRules(JSON.parse(JSON.stringify(data.effective_rules)));
}

// ui/src/pages/RulesConfig.jsx:537-541
const res = await fetch(`/api/v1/rules/${selectedTable}`, {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(tableRules)
});
```

บรรทัด	ทำอะไร
461-464	เปิดตารางแล้วขอ "กฎที่มีผลจริง" (effective_rules = _default รวมกับของตาราง) มาเก็บเป็นสถานะฟอรั่ม
467	เก็บสำเนาไว้เทียบก่อน/หลังตอนกดบันทึก
537-541	ตอนกดบันทึก ส่ง ทั้งก่อน tableRules ไปด้วย PUT ไม่ใช่เฉพาะช่องที่แก้

ขั้นที่ 2: ฝั่ง api รวมเข้ากับส่วนของตารางแล้วเขียนไฟล์

```
# api/app/api/dynamic_rules.py:682-690
config = _load_rules_config()

# Merge new values into the existing table section (create if absent)
existing = config.get(table_name, {})
before_snapshot = copy.deepcopy(existing)
existing.update(body)
config[table_name] = existing

_save_rules_config(config)
```

บรรทัด	ทำอะไร
682	อ่านไฟล์ตั้งค่าทั้งไฟล์
685-688	เอาส่วนของตารางเดิม (ถ้ามี) มา update ด้วยก่อนที่ส่งมา แล้วใส่กลับ
690	เขียนไฟล์ (สำรองไว้ก่อน เก็บสูงสุด 10 ชุด api/app/api/dynamic_rules.py:126-145)

เพราะฟอร์มส่งกฎที่ **รวมค่าเริ่มต้นแล้ว** กลับมาทั้งก้อน การแก้เพียงช่องเดียวจะ **คัดลอกค่าเริ่มต้นทั้งหมดไปเป็นส่วนของตารางนั้นถาวร** หลังจากนั้นการแก้ `_default` จะไม่มีผลกับตารางนี้อีก ผลกระทบนี้เห็นได้จากโครงสร้างโค้ดสองจุดข้างบน (ฟอร์มส่ง `effective_rules` ที่รวมแล้ว, api ใช้ `existing.update(body)`) ยังไม่ได้ทดสอบด้วยการบันทึกจริง เพราะรายงานนี้ห้ามเปลี่ยนค่าในระบบ

ขั้นที่ 3: สำเนาของ Elasticsearch

```
# api/app/api/dynamic_rules.py:159-168
# Sync to ES sdoqap_rules_registry index
try:
    es = _get_es()
    if not es.indices.exists(index="sdoqap_rules_registry"):
        es.indices.create(index="sdoqap_rules_registry")

    for table_name, table_rules in config.items():
        if table_name == "_comment":
            continue
        es.index(index="sdoqap_rules_registry", id=table_name, document=table_rules)
```

ทุกครั้งที่บันทึก ระบบเขียน **ทุกตาราง** ในไฟล์ลง ES ใหม่ (วนตาม `config.items()`) ถ้าเขียน ES ไม่สำเร็จ แต่เขียนไฟล์สำเร็จ การบันทึกก็ยังนับว่าสำเร็จ (`api/app/api/dynamic_rules.py:170-177`)

ขั้นที่ 4: Spark อ่านกฎตอนเริ่มรอบถัดไป

```
# spark/spark_quality_engine.py:1506-1532
parsed = urlparse(ELASTICSEARCH_URL)
auth = (parsed.username, parsed.password) if parsed.username else None
base_url = f"{parsed.scheme}://{parsed.hostname}:{parsed.port}"

# 1. Try reading from ES
es_default = {}
es_table = {}
es_read_success = False

try:
    # Read default config
    url_default = f"{base_url}/sdoqap_rules_registry/_doc/_default"
    res_default = requests.get(url_default, auth=auth, timeout=3)
    if res_default.status_code == 200:
        es_default = res_default.json().get("_source", {})
        es_read_success = True

    # Read table-specific overrides
    url_table = f"{base_url}/sdoqap_rules_registry/_doc/{table_name}"
    res_table = requests.get(url_table, auth=auth, timeout=3)
    if res_table.status_code == 200:
        es_table = res_table.json().get("_source", {})
        es_read_success = True

    if es_read_success:
        print(f"[RULES] Successfully loaded configuration for '{table_name}' from Elasticsearch sdoqap_rules_registry.")
        return _merge_configs_dict(es_default, es_table)
```

บรรทัด	ทำอะไร
1506-1508	สร้างที่อยู่อ ES จากตัวแปร <code>ELASTICSEARCH_URL</code> (ที่ Spark สร้างเองได้จากตัวแปรย่อย <code>spark/spark_quality_engine.py:62-74</code>)

บรรทัด	ทำอะไร
1517-1521	อ่านเอกสาร _default จาก ES
1524-1528	อ่านเอกสารของตารางจาก ES
1530-1532	ถ้าอ่าน ES ได้อย่างน้อยหนึ่งเอกสาร ใช้ ES แล้ว ไม่ดูไฟล์อีก

ถ้าอ่าน ES ไม่ได้เลย จึงกลับไปอ่านไฟล์ (spark/spark_quality_engine.py:1536-1548) ผลคือ **ES เป็นแหล่งที่จริงกว่าไฟล์** ถ้าใครแก้ไฟล์ rules_config.json ตรงๆ โดยไม่ผ่าน api ค่านั้นจะไม่มีผล ตรงที่ ES ยังมีเอกสารของตารางนั้น

```
# spark/spark_quality_engine.py:1550-1574
def _merge_configs_dict(default: dict, table_rules: dict) -> dict:
    """Helper to deep merge table-specific configs override default configs."""
    merged = {}
    for key in set(list(default.keys()) + list(table_rules.keys())):
        default_val = default.get(key)
        table_val = table_rules.get(key)
        if table_val is not None:
            if isinstance(default_val, dict) and isinstance(table_val, dict):
                merged[key] = {**default_val, **table_val}
            else:
                merged[key] = table_val
        elif default_val is not None:
            merged[key] = default_val
    return merged

def resolve_rule_value(rule_entry, fallback):
    """Extract the effective scalar value from a rule entry.
    Supports both v1 flat format (e.g., 90.0) and v2 nested format (e.g., {'mode': 'adaptive', 'base_value': 90.0}).
    Returns the base_value from nested format, or the raw value from flat format."""
    if isinstance(rule_entry, dict):
        return rule_entry.get("base_value", fallback)
    if rule_entry is not None:
        return rule_entry
    return fallback
```

บรรทัด	ทำอะไร
1553-1560	รวมทีละคีย์: ถ้าทั้งสองฝั่งเป็นกล่อง (dict) ให้ส่วนของตารางทับเฉพาะช่องที่ระบุ ไม่เช่นนั้นส่วนของตารางทับทั้งคีย์
1561-1562	คีย์ที่มีเฉพาะใน _default ก็ถูกเก็บไว้
1566-1574	resolve_rule_value ดึงตัวเลขเดี่ยวออกจากกล่องกฎ ถ้าเป็นกล่องจะคืน base_value เสมอ ไม่สนใจ mode

สิ่งที่หน้าจอไม่ได้บอก

- ปุ่มยืนยันบนหน้าจอเขียนว่า "This will immediately affect all incoming ingestion pipelines" (ui/src/pages/RulesConfig.jsx:532) จริงๆ มีผลเฉพาะรอบที่รันหลังจากนี้ ข้อมูลที่ตรวจไปแล้วไม่ถูกตรวจใหม่
- หน้าจอขอกฎด้วย GET /api/v1/rules/{table} ซึ่งตอบ ค่าที่ตั้งไว้ (รวมแล้ว) ไม่ใช่ ค่าที่ Spark ใช้จริงในรอบล่าสุด (api/app/api/dynamic_rules.py:613-634) สองอย่างนี้ต่างกันได้ ดูข้อ 3 เรื่องเกณฑ์ adaptive

ข้อ 3: ตัวเลขในกฎถูกเอาไปคำนวณยังงัย

3.1 เกณฑ์คะแนนคุณภาพ (quality_score_threshold)

Spark เตรียมกฎก่อนตรวจข้อมูล:

```
# spark/spark_quality_engine.py:1711-1734
rules = load_rules_config(table_name)
rules["table_name"] = table_name

# Try to apply dynamic adaptive rules (Layer 2: Statistical Engine)
# Falls back gracefully to base values if dynamic_rules_engine is unavailable
#
# Note: the raw DataFrame isn't loaded yet at this point in the run, so only the
# quality_score_threshold branch (which queries ES history, not the DataFrame) is
# active here – that's the field every table's rules_config.json actually sets to
# "adaptive" mode. null_checks tolerance-learning has no consumer anywhere else in
# this file (unlike value_range, which is computed independently further down at
# the point value_range_profile is built) – it remains unimplemented; treat
# rules_config.json's null_checks.mode as informational until that's built.
try:
    from dynamic_rules_engine import apply_adaptive_rules
    rules = apply_adaptive_rules(rules, table_name, df=None, spark=spark)
```

```

print(f"[DYNAMIC RULES] Adaptive rules applied for '{table_name}')"
except ImportError:
    print(f"[DYNAMIC RULES] dynamic_rules_engine not available, using base config")
except Exception as dre:
    print(f"[DYNAMIC RULES] Failed to apply adaptive rules: {dre}. Using base config.")

quality_threshold = resolve_rule_value(rules.get("quality_score_threshold"), 90.0)
freshness_limit_hours = resolve_rule_value(rules.get("freshness_threshold_hours"), 48)

```

บรรทัด	ทำอะไร
1711-1712	โหลดกฎของตาราง
1726	เรียก <code>apply_adaptive_rules</code> โดยส่ง <code>df=None</code> (ยังไม่มีข้อมูล) ดังนั้นเฉพาะกฎเกณฑ์คะแนนคุณภาพเท่านั้นที่คำนวณใหม่ได้ ตามที่ความคิดเห็นในโค้ดยอมรับเอง (<code>spark/spark_quality_engine.py:1717-1723</code>)
1728-1731	ถ้านำเข้าโมดูลไม่ได้ หรือการคำนวณล้มเหลว พิมพ์ข้อความลง log แล้วใช้ค่าฐานต่อ ไม่มีสถานะใดบอกในผลรันว่าเกิดเหตุนี้
1733	ตัวเลขเกณฑ์ที่ใช้จริง = <code>base_value</code> ของกล่องกฎที่ได้กลับมา

ถ้าโหมดเป็น adaptive ฟังก์ชันคำนวณเกณฑ์จากประวัติ:

```

# spark/dynamic_rules_engine.py:336-349
if len(values) < 2:
    print(f"[DYNAMIC_RULES] Insufficient history ({len(values)} runs). "
          f"Using base_value={base_value}.")
    return float(base_value)

import statistics
moving_avg = statistics.mean(values)
std_dev = statistics.stdev(values)
adaptive = max(moving_avg - std_dev, min_value)

print(f"[DYNAMIC_RULES] Adaptive threshold for '{table_name}': "
      f"avg={moving_avg:.2f}, std={std_dev:.2f}, "
      f"threshold={adaptive:.2f} (base={base_value})")
return round(adaptive, 4)

```

บรรทัด	ทำอะไร
336-339	มีประวัติน้อยกว่า 2 รอบ ใช้ <code>base_value</code>
341-343	ค่าเฉลี่ย และส่วนเบี่ยงเบนมาตรฐานแบบ ตัวอย่าง (หารด้วย $n - 1$) ของคะแนนคุณภาพในหน้าต่างล่าสุด (ค่าเริ่มต้น 15 รอบ <code>spark/dynamic_rules_engine.py:309-315</code>)
344	$\text{เกณฑ์} = \max(\text{ค่าเฉลี่ย} - \text{ส่วนเบี่ยงเบน}, \text{min_value})$ เมื่อ <code>min_value</code> ค่าเริ่มต้นคือ 70
349	ปัดเป็น 4 ตำแหน่ง

ผลการพิสูจน์ (สำคัญ): เกณฑ์ `adaptive` ไม่เคยทำงานจริงในระบบนี้

- ฟังก์ชันนี้อ่านที่อยู่ ES จากตัวแปรแวดล้อม `ELASTICSEARCH_URL` โดยตรง ถ้าไม่มีตัวแปรนี้ จะคืน `base_value` แทนที่โดยไม่ค้นประวัติ (`spark/dynamic_rules_engine.py:76-83, :301-305`)
- ตรวจในคอนเทนเนอร์จริง: `spark-master` และ `spark-worker` ไม่มี `ELASTICSEARCH_URL` (มีแต่ใน `api`) ส่วนตัวรันหลัก `spark_quality_engine.py` ไม่ติดปัญหานี้ เพราะมันประกอบที่อยู่เองจากตัวแปรย่อยได้ (`spark/spark_quality_engine.py:62-74`) จึงเขียนผลลง ES ได้ปกติ
- งาน Spark ถูกส่งผ่านตัวเดมอนที่รันใน `spark-master` (พอร์ต 8099) ไม่ได้รันในคอนเทนเนอร์ `api`: `api` เรียก `http://spark-master:8099/retry` (`api/app/api/pipeline.py:204-208, :293-297`) และไฟล์เดมอนถูกคัดลอกเข้าอิมเมจ Spark (`spark/Dockerfile:44`) จึงใช้ตัวแปรแวดล้อมของ `spark-master` **ขอยกเว้น:** ถ้าเดมอนติดต่อไม่ได้ `api` จะสำรองด้วยการรัน `python spark/spark_quality_engine.py` ในคอนเทนเนอร์ของตัวเอง (`api/app/api/pipeline.py:221, :313`) ซึ่ง `ELASTICSEARCH_URL` ถูกตั้งไว้ ทางนั้นเกณฑ์ `adaptive` น่าจะทำงาน (อนุมานจากโค้ด ยังไม่ได้รัน) แต่ผลรันที่บันทึกไว้ทั้งหมดบ่งว่าใช้ทางเดมอน
- รันฟังก์ชันจริงในคอนเทนเนอร์ `spark-master` กับตาราง `gov_data` (ผลดิบใน `docs/transform-report/evidence/spark-adaptive-threshold-probe.txt`):

กรณี	ผลที่คืน
A. ตามที่ deploy อยู่ (ไม่มี <code>ELASTICSEARCH_URL</code>)	90.0 พร้อมข้อความ "ELASTICSEARCH_URL not set. Using base_value as threshold."
B. ระบุ <code>ELASTICSEARCH_URL</code> ตอนเรียก	100.0 (avg=100.00, std=0.00)

- ผลการรันจริงที่ระบบบันทึกไว้สอดคล้องกับกรณี A: ทั้ง 86 รอบของ `gov_data` เก็บ `effective_quality_threshold` = 90.0 และอีก 8 ตารางที่มีฟิลด์นี้ก็เก็บแต่ค่ากลม 90.0 หรือ 85.0 ไม่มีรอบไหนบันทึกค่าที่มีเศษทศนิยมแบบที่สูตร `adaptive` จะให้ (ดึงจาก `sdoqap_quality_runs` แยกตามตาราง ดูผลรวมใน `docs/transform-report/evidence/spark-threshold-by-table.json`)
- ยืนยันว่าโค้ดในคอนเทนเนอร์ตรงกับ repo: ค่า checksum ของ `dynamic_rules_engine.py`, `spark_quality_engine.py` และ `rules_config.json` เหมือนกันทั้งสองฝั่ง

คำนวณด้วยมือจากสูตรที่ `spark/dynamic_rules_engine.py:341-349` (รายละเอียดครบใน `spark-adaptive-threshold-worked.txt`)

```
gov_data: คะแนน 15 รอบล่าสุดเท่ากับ 100.0 ทุกรอบ
ค่าเฉลี่ย = 100.0 ; ส่วนเบี่ยงเบน = 0.0
เกณฑ์ = max(100.0 - 0.0, 70) = 100.0      (ตรงกับกรณี B; ค่าฐานคือ 90.0)

ตัวอย่างสมมติ 5 รอบ (ไม่ใช่ข้อมูลระบบ): 95, 90, 100, 85, 100
```

ค่าเฉลี่ย = 94.0 ; ผลรวมกำลังสองของส่วนต่าง = 170 ; ความแปรปรวน = 170/4 = 42.5
ส่วนเบี่ยงเบน = 6.5192 ; เกณฑ์ = max(94.0 - 6.5192, 70) = 87.4808

พฤติกรรมของสูตรที่ควรรู้ (ถ้าเปิดให้ทำงานจริง)

- 1. เกณฑ์ ไม่ถูกจำกัดด้วยค่าฐาน gov_data จะได้ 100.0 สูงกว่าค่าฐาน 90.0 ทำให้รอบที่ได้ 99.9% ถูกแจ้งเตือนว่าตก
- 2. ตารางที่คะแนนต่ำคงที่จะ "ปรับตัว" ตามคะแนนต่ำนั้น เช่น benchmark_test ได้ 74.07% ทุกรอบ (20 จาก 27 แถว) เกณฑ์จะกลายเป็น 74.07% แทนที่จะเตือนต่อไปว่าต่ำกว่า 90%
- 3. จากข้อ 2 เมื่อปัด 4 ตำแหน่ง เกณฑ์เป็น 74.0741 ในขณะที่คะแนนจริงคือ 74.07407... จึงเข้าเงื่อนไข "ต่ำกว่าเกณฑ์" ที่ spark/spark_quality_engine.py:2577 แม้ผลคงที่ทุกรอบ (คำนวณด้วยมือ ไม่ได้สังเกตในระบบจริง เพราะกรณี A ใช้อยู่)

3.2 การตรวจระดับแถว

ค่าว่างทุกคอลัมน์ (คีย์หลักว่างตรวจก่อนหน้านี้ที่ spark/spark_quality_engine.py:2062-2072 เหตุผล missing_primary_key)

```
# spark/spark_quality_engine.py:2074-2088
# — Null Validation: Check all schema columns for null values —
non_pk_cols = [c for c in schema_spec.keys() if c not in pk_cols and c in df.columns]
for col_name in non_pk_cols:
    null_reason = f"null_value_in_{col_name}"
    df_with_status = df_with_status.withColumn(
        "reject_reason",
        F.when(
            (~F.col("is_invalid")) & F.col(col_name).isNull(),
            F.when(F.col("reject_reason") == F.lit(""), F.lit(null_reason))
                .otherwise(F.concat(F.col("reject_reason"), F.lit("; "), F.lit(null_reason)))
        ).otherwise(F.col("reject_reason"))
    ).withColumn(
        "is_invalid",
        F.col("is_invalid") | F.col(col_name).isNull()
    )
```

บรรทัด	ทำอะไร
2075	เอาทุกคอลัมน์ในสคีม่า ที่ไม่ใช่คีย์หลัก
2076-2077	วนทีละคอลัมน์ เตรียมข้อความ null_value_in_<คอลัมน์>
2078-2084	ถ้าแถวยังไม่ถูกตัดว่าเสีย และคอลัมน์นี้ว่าง ให้ต่อข้อความเหตุผลเข้าไป (ต่อด้วย ; ถ้ามีเหตุผลเดิมอยู่แล้ว)

บรรทัด	ทำอะไร
2085-2088	แกวกลายเป็นเสีย (is_invalid) ถ้าคอลัมน์ใดคอลัมน์หนึ่งว่าง

ข้อนี้ไม่ใช่ค่าจากกฎ null_checks เลย ค่าว่างในคอลัมน์ใดของสคีมาก็ทำให้แถวถูกกักกันเสมอ ไม่ว่าฟอร์มจะเลือก Strict หรือ Adaptive (ดูตาราง "ตั้งค่าไหนมีผลจริง")

ลบแถวซ้ำก่อนตรวจ (auto-clean)

```
# spark/spark_quality_engine.py:2043-2060
    non_null_pk_cond = F.col(primary_key).isNotNull() if isinstance(primary_key, str) else F.col(pk_cols[0]).isNotNull()
    df_non_null = df.filter(non_null_pk_cond)
    df_null_pk = df.filter(~non_null_pk_cond)

    df_count_before = df_non_null.count()
    if date_column and date_column in df.columns:
        df_non_null = df_non_null.orderBy(F.col(date_column).desc())

    df_non_null_dedup = df_non_null.dropDuplicates(subset=pk_cols)
    df_count_after = df_non_null_dedup.count()

    dup_resolved = df_count_before - df_count_after
    if dup_resolved > 0:
        print(f"[AUTO-CLEAN] Deduplicated and resolved {dup_resolved} duplicate records.")
        remediation_logs.append(f"resolved_{dup_resolved}_duplicates")

    # Re-combine non-null deduped rows with null PK rows to preserve data integrity
    df = df_non_null_dedup.unionByName(df_null_pk, allowMissingColumns=True)
```

บรรทัด	ทำอะไร
2043-2045	แยกแถวที่คีย์หลักไม่ว่างออกจากแถวที่ว่าง
2047-2049	นับจำนวนก่อน และเรียงตามวันที่จากใหม่ไปเก่า (ถ้ามีคอลัมน์วันที่)
2051	dropDuplicates ตามคีย์หลัก เก็บหนึ่งแถวต่อคีย์
2054-2057	ถ้ามีแถวถูกลบ บันทึกแค่ข้อความ resolved_N_duplicates ในผลการรัน
2060	รวมแถวคีย์ว่างกลับเข้าไปเพื่อให้ไปโดนกักกันในขั้นถัดไป

ผลของขั้นนี้ (ค่าเริ่มต้น auto_clean = true ที่ spark/spark_quality_engine.py:2031):

- แถวซ้ำ หายไปเจียบๆ ไม่ถูกกักกัน และไม่นับในคะแนน เพราะ total_records นับจาก Clean + Quarantine หลังลบแล้ว (spark/spark_quality_engine.py:2358-2360)
ตัวอย่างจริง: grocery_sales มี resolved_1_duplicates ในทุกรอบ แต่ quarantine_breakdown มีแต่ null_value_in ยอดขายรวม: 4
- โค้ดเรียงตามวันที่ก่อน dropDuplicates เพื่อหวังเก็บแถวล่าสุด แต่การรับประกันว่า Spark จะเก็บแถวไหนหลัง dropDuplicates ขึ้นกับ Spark เอง ไม่ใช่ตัวโค้ดนี้ (ยังไม่ได้ทดสอบว่าเก็บแถวไหนจริง)
- การตรวจ duplicate_records (แถว 6 ในตาราง ข้อ 1) จึงแทบไม่มีอะไรให้จับเมื่อ auto_clean เปิด

คำผิดปกติแบบ IQR

```
# spark/spark_quality_engine.py:2236-2264
vr_config = rules.get("value_range", {})
vr_mode = vr_config.get("mode", "off") if isinstance(vr_config, dict) else "off"

if vr_mode in ("auto", "adaptive"):
    # Identify numeric columns from schema_spec
    numeric_cols = [col for col, t in schema_spec.items()
                     if t in ("IntegerType", "DoubleType") and col in clean_df.columns]

    if numeric_cols:
        iqr_mult = vr_config.get("iqr_multiplier", 1.5) if isinstance(vr_config, dict) else 1.5
        value_range_profile = compute_value_range_rules(clean_df, numeric_cols, multiplier=iqr_mult)

        if value_range_profile:
            flagged_df = flag_outlier_rows(clean_df, value_range_profile)
            outlier_df = flagged_df.filter(F.col("_outlier_flag") == True) \
                                  .withColumn("is_invalid", F.lit(True)) \
                                  .withColumn("reject_reason", F.col("_outlier_details")) \
                                  .drop("_outlier_flag", "_outlier_details")
            clean_df = flagged_df.filter((F.col("_outlier_flag") == False) | F.col("_outlier_flag").isNull()) \
                                .drop("_outlier_flag", "_outlier_details")

            outlier_count = outlier_df.count()
            if outlier_count > 0:
                print(f"[DYNAMIC RULES] IQR outlier detection flagged {outlier_count} rows as value outliers")
                remediation_logs.append(f"iqr_outliers_flagged_{outlier_count}")
except ImportError:
    pass # dynamic_rules_engine not available, skip outlier detection
```

```
except Exception as ore:
    print(f"[DYNAMIC RULES] IQR outlier detection failed: {ore}. Continuing without outlier flagging.")
```

บรรทัด	ทำอะไร
2237	อ่าน <code>value_range.mode</code> (ไม่มีให้ถือเป็น <code>off</code>)
2239	ทำงานเมื่อโหมดเป็น <code>auto</code> หรือ <code>adaptive</code> สองโหมดนี้ทำงานเหมือนกันทุกประการ
2241-2242	ใช้ ทุกคอลัมน์ตัวเลข (Integer/Double) ในสคิมา รวมถึงคอลัมน์ <code>id</code> หรือลำดับ
2245	ตัวคูณเดี่ยวสำหรับทุกคอลัมน์ ค่าเริ่มต้น 1.5
2246	คำนวณรั้วของแต่ละคอลัมน์ (<code>compute_value_range_rules</code>)
2249-2255	แถวที่พ้นรั้วของคอลัมน์ใดก็ได้ ไปกักกัน โดยเหตุผลคือข้อความรายละเอียดที่ระบุค่าและช่วงที่คาด แถวที่เหลือไปต่อ

รั้วคำนวณจาก `approxQuantile` ซึ่งเป็นค่าประมาณ (ความคลาดเคลื่อน 1%) ไม่ใช่ตำแหน่งที่แน่นอน (`spark/dynamic_rules_engine.py:239`) ต่างจากฟังก์การ์ดที่ใช้ตำแหน่งที่แน่นอนของ `pandas`

คำนวณด้วยมือจากสูตรที่ `spark/dynamic_rules_engine.py:244-247` (ข้อมูลจริง `grocery_sales` คอลัมน์ `ราคาต่อหน่วย`)

```
Q1 = 25.0 ; Q3 = 50.0 ; ตัวคูณ = 1.5
IQR      = 50.0 - 25.0 = 25.0
รั้วล่าง  = 25.0 - 1.5 * 25.0 = -12.5      (ระบบบันทึกไว้ -12.5)
รั้วบน    = 50.0 + 1.5 * 25.0 = 87.5      (ระบบบันทึกไว้ 87.5)
```

ข้อควรระวังจากข้อมูลจริง: โปรไฟล์ IQR ของ `gov_data` มีรั้วสำหรับคอลัมน์ `id` และ `ลำดับ` (`Q1 = 3, Q3 = 9, รั้ว -6 ถึง 18`) คอลัมน์แบบหมายเลขรันถูกมองเป็นตัวเลขวัดค่า ถ้าตารางนี้โตขึ้นจนหมายเลขเกิน 18 แถวเหล่านั้นจะถูกกักกันเป็น "ค่าผิดปกติ" ทั้งที่เป็นหมายเลขปกติ (อนุมานจากโค้ดและโปรไฟล์ ยังไม่ได้ทดสอบกับข้อมูลที่โตขึ้น)

ค่าผิดปกติแบบ Z-score (ไม่มีช่องตั้งค่า)

```
# spark/spark_quality_engine.py:2271-2280
numeric_cols = [col for col, t in schema_spec.items()
                 if t in ("IntegerType", "DoubleType") and col in clean_df.columns]

if numeric_cols:
    flagged_unsupervised = detect_unsupervised_anomalies(clean_df, numeric_cols, threshold=3.0)
    unsupervised_outlier_df = flagged_unsupervised.filter(F.col("_unsupervised_anomaly") == True) \
                                                .withColumn("is_invalid", F.lit(True)) \
                                                .withColumn("reject_reason", F.col("_unsupervised_anomaly_details")) \
                                                .drop("_unsupervised_anomaly", "_unsupervised_anomaly_details")
```

```
clean_df = flagged_unsupervised.filter((F.col("_unsupervised_anomaly") == False) | F.col("_unsupervised_anomaly").isNull()) \
    .drop("_unsupervised_anomaly", "_unsupervised_anomaly_details")
```

บรรทัด	ทำอะไร
2271-2272	ใช้ทุกคอลัมน์ตัวเลขเช่นเดียวกับ IQR
2274	เรียกด้วยเกณฑ์ตายตัว 3.0 ไม่อ่านจากกฎใดๆ
2275-2280	แถวที่ Z-score เกิน 3.0 ในคอลัมน์ใดคอลัมน์หนึ่ง ไปกักกัน

สูตรอยู่ที่ spark/dynamic_rules_engine.py:641-643: $\text{abs}(\text{ค่า} - \text{ค่าเฉลี่ย}) / \text{ส่วนเบี่ยงเบน} > 3.0$ คอลัมน์ที่ส่วนเบี่ยงเบนเป็น 0 ถูกข้าม (:637-639) ขั้นนี้ทำงานหลัง IQR จึงคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนจากข้อมูลที่ผ่าน IQR มาแล้ว

ตั้งค่าไหนมีผลจริง

ตารางนี้เทียบทุกช่องที่ผู้ใช้ตั้งได้ (และคัญสำคัญในไฟล์) กับโค้ดที่อ่านมันจริง

คีย์	มีช่องบนฟอร์มใหม่	Spark อ่านและใช้ตัดสินใจ	ผลจริง
quality_score_threshold.base_value	มี (ui/src/pages/RulesConfig.jsx:958-982)	อ่านผ่าน resolve_rule_value (spark/spark_quality_engine.py:1733)	มีผล: ตัดสินการแจ้งเตือน (:2577), สถานะ success/warnings (:2853), และการเรียก Gold layer (:2860)
quality_score_threshold.mode = adaptive	มี (:938-955)	โค้ดคำนวณใหม่ได้ (spark/dynamic_rules_engine.py:413-438)	ไม่มีผลใน deployment นี้: ได้ค่าฐานเสมอ (ข้อ 3.1)
quality_score_threshold.min_value, adjustment_window_runs	ไม่มี (มีค่า 70.0 ผิงในโค้ดฟอร์มตอนสร้างกล่องใหม่ :948)	อ่านเฉพาะในสาขา adaptive	ไม่มีผลเช่นเดียวกับข้างบน
null_checks.mode, default_tolerance	มี (:1035-1042)	สาขา adaptive ต้องมีข้อมูล แต่ส่ง df=None (spark/spark_quality_engine.py:1726) จึงไม่ทำงาน (spark/dynamic_rules_engine.py:444)	ไม่มีผล: ค่าว่างในคอลัมน์ใดของสคีมามากักกันเสมอ (spark/spark_quality_engine.py:2074-2088)
value_range.mode	มี (:1046-1054)	อ่าน (spark/spark_quality_engine.py:2237-2239)	มีผล: off ปิดการตรวจ IQR ส่วน auto และ adaptive เหมือนกัน
value_range.iqr_multiplier	ไม่มี	อ่าน ใช้ตัวเดียวทุกคอลัมน์ (:2245)	มีผล แต่แก้ได้ทาง API/ไฟล์เท่านั้น

ดีย์	มีช่องบนฟอร์มใหม่	Spark อ่านและใช้ตัดสินใจ	ผลจริง
value_range.column_overrides	ไม่มี	ไม่มีโค้ดอ่าน (ค้นทั้ง spark/ และ api/: มีแค่ตัวที่เขียนคือ AI Advisor และ เเทมเพลต)	ไม่มีผล: ค่าตัวอย่างเช่น bank_data_csv ที่ตั้ง age เป็น 2.5 ไม่ถูกใช้
null_primary_key.enabled, duplicate_check.enabled, null_date_column.enabled	ไม่มี	ไม่ถูกอ่านโดยตัวตรวจ (พบเฉพาะในตัวสร้างเทมเพลต spark/dynamic_rules_engine.py:708-710 และเส้นทางข้อเสนอ AI)	ไม่มีผล: การตรวจเหล่านี้ทำงานเสมอ (spark/spark_quality_engine.py:2062-2072, :2112-2121, :2043-2060) ตัวอย่างเช่น ตั้ง enabled: false แล้วก็ยังตรวจ
freshness_threshold_hours	มี (ui/src/pages/RulesConfig.jsx:985-1030)	อ่านค่าไว้ (spark/spark_quality_engine.py:1734) และบันทึกลงผลรัน (:2828)	ไม่มีผลต่อการตัดสินใจ: คำนวณความหน่วงได้จริง (:2537-2561) แต่ไม่มีการเทียบกับเกณฑ์ ไม่พบการเทียบใน api/, ui/src/ หรือ spark/ (พบเฉพาะการเฉลี่ยความหน่วงใน spark/spark_gold_layer.py)
ai_advisor.enabled	มี (:1059-1070)	อ่าน (spark/spark_quality_engine.py:2614)	มีผล: เปิดเป็นการเรียกส่วนวิเคราะห์ทั้งบล็อก
ai_advisor.model	มี (:1072-1079)	ตัวที่ใช้จริงไม่อ่านค่านี้ โมเดลมาจากการตั้งค่าใน ES หรือตัวแปร GROQ_MODEL (spark/ai_rule_advisor.py:181-199) และถ้าชื่อเท่ากับค่าเริ่มต้น llama-3.3-70b-versatile โค้ดจะใช้ openai/gpt-oss-120b แทน (:218) ทั้งที่ข้อความบันทึกยังพิมพ์ชื่อเดิม (:228)	ไม่มีผล และชื่อที่เห็นในฟอร์มไม่ใช่ชื่อที่ถูกเรียก
ai_advisor.trigger, confidence_threshold, max_rows_to_analyze	ไม่มี	อ่าน (spark/spark_quality_engine.py:2638, :2698, :2659)	มีผล แก่ได้ทาง API/ไฟล์เท่านั้น
auto_clean	ไม่มี	อ่าน (spark/spark_quality_engine.py:2031)	มีผล: เปิดอยู่ = ลบแถวซ้ำก่อนตรวจ
Z-score เกณฑ์ 3.0	ไม่มี ไม่มีคีย์	ค่าตายตัว (:2274)	ทำงานเสมอกับทุกตาราง ปิดไม่ได้
induced (กฎที่เรียนรู้)	ไม่มี	อ่านถ้ามี (:2294)	มีผลเมื่อมีกฎถูกอนุมัติเข้ามา

สรุป: ฟอร์มมี 8 ช่อง (โหมด+ค่าเกณฑ์คุณภาพ, โหมด+ความหน่วงข้อมูล, โหมดค่าว่าง, โหมด IQR, เปิด AI, ชื่อโมเดล) ในนั้น มีผลต่อการตัดสินใจจริง 3 ช่อง คือค่าเกณฑ์คุณภาพ โหมด IQR และสวิตช์เปิด AI ส่วนที่เหลือไม่มีผล (ความอดทนต่อค่าว่าง, ความหน่วง, ชื่อโมเดล, และโหมด adaptive ของเกณฑ์คุณภาพ ซึ่งไม่ทำงานในระบบนี้) การนับนี้นับโหมด+ค่าของเกณฑ์คุณภาพเป็นสองช่องที่แยกกัน

ข้อ 4: หลังตรวจแล้ว ข้อมูลไปที่ไหน

แถวที่ไม่ผ่านทุกประเภทไปชนกักกันรวมกัน

```
# spark/spark_quality_engine.py:2350-2360
# Add run_id partition structure to quarantined parquet
all_quarantined_write = all_quarantined.withColumn("run_id", F.lit(run_id)) \
    .withColumn("rejected_at", F.current_timestamp())

# Cache and count before writing to avoid re-reading and partial failures
clean_df.cache()
all_quarantined_write.cache()

clean_count = clean_df.count()
quarantine_count = all_quarantined_write.count()
total_records = clean_count + quarantine_count
```

บรรทัด	ทำอะไร
2351-2352	ดีตรัสรอบ (run_id) และเวลาที่ถูกละทิ้ง (rejected_at) ให้ทุกแถวก็กกัน
2358-2360	นับ Clean, Quarantine และ total_records = ผลรวมของสองอย่าง (แถวซ้ำที่ auto-clean ลบไปแล้วไม่ถูกนับ)

แถวก็กกันถูก **เพิ่มต่อท้าย (append)** ตาราง Delta แยกโฟลเดอร์ตาม run_id (spark/spark_quality_engine.py:2436) จึงเก็บประวัติทุกรอบ ไม่เขียนทับ

แถวที่ผ่านถูกรวม (MERGE) เข้าตาราง active

```
# spark/spark_quality_engine.py:2383-2406
# Delta Lake MERGE (True Upsert)
clean_df_for_upsert = clean_df.withColumn("run_id", F.lit(run_id))

try:
    from delta.tables import DeltaTable
    if DeltaTable.isDeltaTable(spark, active_path):
        print("Existing Delta Table found. Performing MERGE INTO...")
        delta_table = DeltaTable.forPath(spark, active_path)
        if isinstance(pk_cols, list) and len(pk_cols) > 0:
            merge_cond = " AND ".join([f"old.`{col}` = new.`{col}`" for col in pk_cols])
        else:
            merge_cond = f"old.`{pk_cols}` = new.`{pk_cols}`"
        delta_table.alias("old").merge(
            clean_df_for_upsert.alias("new"),
```

```
merge_cond
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
print("Delta Lake Upsert completed successfully.")
else:
    print(f"No existing Delta table. Creating new Delta table at: {active_path}")
    clean_df_for_upsert.write \
        .format("delta") \
        .mode("overwrite") \
        .save(active_path)
    print("Delta Lake table created successfully.")
```

บรรทัด	ทำอะไร
2384	ดีดรอห้สรวบให้แถวที่ผ่าน
2388-2398	ถ้ามีตาราง active อยู่แล้ว: MERGE ตามคีย์หลัก คีย์ตรงให้อัปเดตทั้งแถว คีย์ใหม่ให้เพิ่ม
2400-2406	ถ้ายังไม่มีตาราง: สร้างใหม่ด้วยแถวรวบนี้

ผลที่ตามมา: ตาราง active คือ **สถานะรวมสะสมของทุกรอบ** ไม่ใช่ผลของรอบล่าสุดเท่านั้น ถ้า MERGE ล้มเหลว ระบบหยุดรวบ บันทึทสถานะ failed และไม่แตะตาราง active (spark/spark_quality_engine.py:2407-2420)

คะแนนกับเกณฑ์: ตัดสินการเดินต่อ

```
# spark/spark_quality_engine.py:2567-2582
# 4. QUALITY SCORE
passed_tests = clean_count
total_tests = total_records
if total_tests == 0:
    quality_score = 0.0
    remediation_logs.append("Warning: Empty source file ingested. Quality score defaulted to 0.0% to prevent masking upstream ingestion failure.")
else:
    quality_score = (passed_tests / total_tests) * 100.0

# FIX 3A: Use per-table threshold from rules_config instead of hardcoded 90.0
if quality_score < quality_threshold:
    send_n8n_alert(
        title=f"🔥 Critical Data Quality Drop: {table_name}",
        message=f"Run ID: {run_id}\nQuality Score: {quality_score:.1f}% (threshold={quality_threshold}%) \nQuarantined: {quarantine_count} rows\nCle
```

```
an: {clean_count} rows",
    severity="critical"
)
```

บรรทัด	ทำอะไร
2568-2574	คะแนน = Clean ÷ (Clean + Quarantine) × 100 ถ้าวางให้ 0.0
2577-2582	ถ้าคะแนนต่ำกว่าเกณฑ์ ส่งการแจ้งเตือนวิกฤตผ่าน n8n

คำนวณด้วยมือจากสูตรที่ spark/spark_quality_engine.py:2574 (ข้อมูลจริง grocery_sales)

```
คะแนน = 557 / 561 * 100 = 99.2870%      (ระบบบันทึกไว้ 99.287)
เกณฑ์ที่ใช้ = 90.0  ->  99.287 < 90.0 เป็นเท็จ  ->  ไม่แจ้งเตือน
```

ขั้นต่อจากนี้ขึ้นกับคะแนนเทียบเกณฑ์:

เงื่อนไข	สิ่งที่เกิด
ต่ำกว่าเกณฑ์	แจ้งเตือน n8n (:2577) และสถานะรอบเป็น warnings (:2853)
ตั้งแต่เกณฑ์ขึ้นไป	สถานะรอบเป็น success (:2853) และเรียกสร้าง Gold layer ใหม่ในพื้นที่หลัง (:2860-2870)

สิ่งที่ถูกเขียนลง Elasticsearch ทูกรอบ: sdoqap_quality_runs (คะแนน, จำนวน, เกณฑ์ที่ใช้ effective_quality_threshold, รั้ว IQR value_range_profile, เหตุผลกักกันรวม quarantine_breakdown), sdoqap_lineage_runs และ sdoqap_pipeline_runs (spark/spark_quality_engine.py:2806-2855) ค่าที่ Spark ใช้จริงจึงถูกเก็บไว้แล้ว แต่ หน้าเว็บส่วนใหญ่ไม่แสดง (ดู 03-transform-analysis.md) หนึ่งจุดที่แสดง: เหตุผลกักกันรวมบนแดชบอร์ด (ui/src/pages/Dashboard.jsx:1627-1632)

ตัวช่วยวิเคราะห์ AI (ทำงานหลังคะแนนถูกคำนวณ)

- เงื่อนไขเรียกตัวช่วย: เปิด ai_advisor.enabled และ (trigger เป็น always หรือรอบนี้เป็นค่าผิดปกติทางสถิติ หรือคะแนนต่ำกว่าเกณฑ์เกิน 15 จุด หรือโปรไฟล์ข้อมูลเปลี่ยน) (spark/spark_quality_engine.py:2638-2640)
- ตัวช่วยเริ่มทำงานที่ spark/spark_quality_engine.py:2613-2659 ai_advisor.enabled เปิดเป็นค่าเริ่มต้น (spark/rules_config.json) และ trigger: "always" จึงทำงานทูกรอบ
- ถ้าไม่ได้ตั้งกฎแฉ Groq ระบบใช้ตัววิเคราะห์แบบกฎสำเร็จรูปในเครื่อง (heuristic) แทนโมเดลภาษา (spark/ai_rule_advisor.py:202-204) วิธีที่ใช้ถูกบันทึกในเอกสารข้อเสนอ (analysis_metadata.method, spark/spark_quality_engine.py:2711) ไม่ใช่ในผลการรัน
- ผลของมันคือ ข้อเสนอกฎ (PENDING) ที่ต้องมีคนอนุมัติ ไม่เขียนลงกฎเองอัตโนมัติ (ความคิดเห็นในโค้ด :2700-2709)

ผู้ใช้เห็นผลย้อนหลังได้ที่ไหน: ปุ่ม "ดูข้อมูล" ในหน้า Jobs & Pipelines เรียก GET /api/v1/export/records/{layer}/{table} ซึ่งอ่านตาราง Delta ชั้น active หรือ quarantine แล้วกรองตามรหัสรอบได้ (api/app/api/data_export.py, ฟังก์ชัน search_dataset_records)

เทียบกับกฎ 3 การ์ด

หัวข้อ	3 การ์ด (เอกสาร 01)	กฎรายการ Spark (เอกสารนี้)
ข้อมูลที่ตรวจ	ไฟล์ CSV ที่อัปโหลด 1 ไฟล์ในหน่วยความจำของ api	ข้อมูลใน HDFS ทั้งชุดต่อตาราง หลายตาราง
เมื่อไรที่กฎมีผล	ทันทีทุกครั้งที่แก้ (คำนวณใหม่ทั้งไฟล์)	เมื่อ Spark รันรอบถัดไปเท่านั้น
ลำดับการตรวจ	ซ้ำ → วาง/นอกช่วง → ค่าผิดปกติ	ลบซ้ำ → คีย์ว่าง → วางทุกคอลัมน์ → ชนิด → วันที่ → IQR → Z-score
คอลัมน์ที่ถูกตรวจ	score และ study_hours เท่านั้น	ทุกคอลัมน์ในสคีมา (วาง), ทุกคอลัมน์ตัวเลข (IQR, Z-score)
วิธีหาค่าผิดปกติ	รั้ว Tukey บนคอลัมน์เดียว ตำแหน่งแม่นยำ ค่า k เลือก 3.0/1.5	IQR ตัวคูณ 1.5 บนทุกคอลัมน์ตัวเลข ตำแหน่งประมาณ (1%) + Z-score 3.0 ดายตัว
ค่าผิดปกติไปที่	Review (รอคนตัดสินใจ)	Quarantine (กักกันทันที ไม่มีโซน Review)
แถวซ้ำ	ตัดสินใจตามคีย์ที่เลือก ไปกักกัน (หรือ Review)	ถูกลบเจียบๆ ก่อนตรวจ ไม่นับในคะแนน (เมื่อ auto_clean เปิด)
ค่าว่าง	ตั้งนโยบายได้ (strict / adaptive_5) แต่ไม่มีช่องบนหน้า	กักกันเสมอทุกคอลัมน์ ช่อง null_checks ไม่มีผล
เหตุผลรายแถว	ป้ายข้อความคงที่ (ไม่มีค่าจริง)	รวมค่าจริงและช่วงที่คาดสำหรับ IQR/Z-score
ผลลัพธ์ไปที่	3 ไฟล์ CSV ในโฟลเดอร์ api (เขียนทับทุกครั้ง)	Delta บน HDFS: active (MERGE สะสม), quarantine (append ตามรอบ) + ผลรันใน Elasticsearch
ประวัติ	ไม่มี	ถูกรอบเก็บใน sdoqap_quality_runs

ข้อที่ต่างจากที่คาด

รายการนี้บันทึกสิ่งที่ตรวจโค้ดหรือรันจริงแล้วต่างจากข้อสันนิษฐานก่อนหน้านี้ (รวมข้อสรุปในแชทก่อนหน้านี้) หรือเป็นของใหม่ที่เจาะระหว่างเขียน

- จากที่เคยตอบในแชท: เคยตอบว่าเกณฑ์ที่ใช้จริงในแต่ละรอบ "ไม่ได้ถูกบันทึกไว้" ซึ่งผิด ระบบบันทึก effective_quality_threshold, value_range_profile และ quarantine_breakdown ไว้ในผลรันทุกรอบ (spark/spark_quality_engine.py:2823-2830) จุดบอดคือหน้าเว็บไม่แสดง ไม่ใช่ข้อมูลหาย
- จากที่เคยตอบในแชท: เคยตอบว่า value_range แบบ adaptive ตั้งแล้วไม่มีผล ซึ่งผิด ตัวตรวจ IQR ทำงานจริง (ข้อ 3.2) ช่องที่ไม่มีผลคือ null_checks
- เกณฑ์ adaptive ไม่ทำงานจริง เพราะตัวแปร ELASTICSEARCH_URL ไม่มีในคอนเทนเนอร์ Spark ยืนยันด้วยการรันฟังก์ชันจริง 2 กรณี และประวัติ 86 รอบของ gov_data (ข้อ 3.1)
- เกณฑ์ adaptive อาจสูงกว่าค่าฐานและตามคะแนนต่ำคงที่ ถ้าซ่อมข้อ 3 โดยไม่แก้สูตร gov_data จะได้เกณฑ์ 100.0 (พิสูจน์ด้วยกรณี B) และตารางที่ต่ำคงที่จะหยุดถูกเตือน
- ฟอร์มส่งกฎที่รวมแล้วทั้งก่อน การบันทึกช่องเดียวอาจคัดลอกค่าเริ่มต้นทั้งหมดไปเป็นส่วนของตารางนั้น (อนุมานจากโค้ด ยังไม่ได้ทดสอบเพราะห้ามเปลี่ยนค่าในระบบ)
- แถวซ้ำหายไปเจียบๆ เมื่อ auto_clean เปิด ไม่ถูกกักกัน ไม่นับในคะแนน มีร่องรอยเพียงข้อความ resolved_N_duplicates
- สวิตช์ *.enabled และ column_overrides ไม่มีโค้ดอ่าน ตั้งเป็น false หรือเพิ่มรายการแล้วไม่มีอะไรเปลี่ยน
- เกณฑ์ความหน่วงข้อมูลมีอยู่แต่ไม่ถูกเทียบกับอะไรเลย ทั้งที่ฟอร์มมีช่อง "Max Allowed Delay (Hours)"
- ชื่อโมเดล AI บนฟอร์มไม่ถูกใช้ และโค้ดสลับเป็น openai/gpt-oss-120b เมื่อชื่อเป็นค่าเริ่มต้น
- IQR และ Z-score ครอบคลุมคอลัมน์ id/ลำดับ เห็นในโปรไฟล์จริงของ gov_data

11. ผลรันของ `gov_data` และ `benchmark_test` เป็นข้อมูลซ้ำๆ ทุกรอบให้คะแนนเท่ากัน (100.0 และ 74.07) ค่าเฉลี่ยจึงมีส่วนเบี่ยงเบน 0 ตัวอย่างที่มีความแปรปรวนจริงจึงใช้ชุดสมมติที่ระบุชัดเจนในไฟล์หลักฐาน
12. ฟิวด์ `rules_mode` ในผลรันบอกโครงสร้างของตั้งค่า ไม่ใช่สิ่งที่เกิดขึ้น: ตั้งเป็น `adaptive` เมื่อถูกเป็นกล่อง (`spark/spark_quality_engine.py:2826`) ทุกรอบของ `gov_data` จึงเขียน `adaptive` ทั้งที่ใช้เกณฑ์ฐาน 90.0 ผู้ดูแลจึงถูกชวนให้เข้าใจว่าเกณฑ์ปรับตัวแล้ว
13. ใน **ES** มีข้อมูลตัวอย่างปนอยู่: `api/seed_es.py` เขียนเอกสารผลรันสมมติลงดัชนีเดียวกัน (เช่น บรรทัด 80-149) รายงานนี้ใช้เฉพาะเอกสารที่มีฟิวด์ `effective_quality_threshold` ซึ่งมีแต่เอนจิน Spark ที่เขียน (`spark/spark_quality_engine.py:2827`)

ส่วนที่ 2: วิเคราะห์ Transform — ควรปรับอะไร ควรเน้นอะไร

เอกสารนี้ตอบโจทย์ส่วนที่ 2 ของเพื่อน และข้อกำหนดเพิ่มเติมที่ว่า ในหน้าที่เกี่ยวกับ Transform ให้หาว่าจุดไหนดูเหมือน **black box** แล้วทำให้เป็น **white box** โดยไม่ใช่การเขียนคำอธิบายเพิ่มหรือเนื้อหาให้อ่านลงในเว็บ ข้อเท็จจริงทุกข้อมาจากเอกสาร 01-interactive-rules.md, 02-spark-table-rules.md และหลักฐานใน docs/transform-report/evidence/ (รหัส S1-S10 คือสถานการณ์ในไฟล์ interactive-scenarios.json)

นิยาม black box / white box ที่ใช้ในรายงานนี้

- Black box:** จุดที่ผู้ใช้เปลี่ยนค่าหรืออ่านผลแล้ว ไม่สามารถเห็นจากระบบเอง ว่าระบบทำอะไรไป เช่น ตั้งค่าแล้วไม่รู้ว่าค่าไปมีผลตรงไหน ตัวเลขบนจอมาจากไหน แถวนี่ถูกตัดสินใจเพราะกฎข้อไหนและค่าจริงเท่าไร หรือช่องที่ตั้งได้แต่ไม่มีผลเลย
- White box:** ผู้ใช้ตามรอยได้ครบสาย ค่ากฎ → การคำนวณ → แถวที่โดน → ไฟล์ผลลัพธ์ ด้วยสิ่งที่ระบบแสดงหรือบันทึกเอง
- ข้อกำหนดของงานนี้:** ห้ามแก้ด้วยการเพิ่มข้อความอธิบาย เช่น ไอคอน ? ข้อความช่วยเหลือ แบนเนอร์ หรือหน้าคู่มือ วิธีที่ยอมรับคือ ให้ระบบแสดงการทำงานของตัวเองออกมาด้วยกราฟ ตัวอย่างผลก่อนใช้จริง การเจาะลงไปดูแถว คอลัมน์ข้อมูลแบบแยกช่อง การบันทึกค่าที่ใช้จริง หรือการเอาสิ่งปลอมออก
- หมายเหตุเรื่องงานก่อนหน้า:** ไอคอน ? (InfoHint) และแบนเนอร์ที่เพิ่มในรอบแก้ปัญาก่อนหน้า เป็น "ข้อความ" จึงไม่นับ เป็นการทำให้เป็น white box ตามข้อกำหนดนี้ ทีมต้องตัดสินใจว่าจะเก็บหรือเอาออกเมื่อทำแบบใหม่เสร็จ

6 วิธีทำ white box โดยไม่เพิ่มข้อความ

รหัส	วิธี	หน้าตาเมื่อทำเสร็จ
A	แสดงการคำนวณให้เห็น	กราฟข้อมูลจริง (เช่น study_hours) พร้อมเส้น Q1, Q3 และรั้ว ที่ขยับตามค่าที่ตั้ง
B	แสดงหลักฐาน	กดที่ตัวเลข "ติด 65 แถว" แล้วเห็นแถวเหล่านั้นจริง
C	แสดงผลกระทบก่อนใช้จริง	เปลี่ยนค่าเป็น "ร่าง" เห็นว่าแถวจะย้ายโซนก็แถวก่อนกดยืนยัน
D	แสดงร่องรอยรายแถว	ทุกแถวมีช่อง กฎ / คอลัมน์ / ค่าจริง / เงื่อนไข / เกณฑ์ แยกกัน
E	แสดงค่าที่ระบบใช้จริง	บันทึก "ใบบันทึกการรัน" (manifest) ของแต่ละรอบ แล้วแสดงเทียบกับค่าที่ตั้งไว้
F	เอาของปลอมออก	ตัวเลขตายตัวให้คำนวณจริง ช่องที่ตั้งแล้วไม่มีผลให้ปิดหรือเอาออก

ขนาดงาน (ประมาณการ ไม่ได้วัดจากการลงมือ): **S** = ไม่เกินครึ่งวัน, **M** = ครึ่งวันถึง 2 วัน, **L** = มากกว่า 2 วัน

จุด black box แยกตามหน้า

Data Ingestion

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประเภท	ขนาด	ไฟล์ที่ต้องแก้
BB-01	ไฟล์ที่ไม่มีคอลัมน์ student_id, course, score, study_hours ครบ จะไม่ถูกส่งเข้ากฎเลย (เก็บเป็นไฟล์แยกและทำแค่สรุปสถิติ)	api/app/api/whitebox.py:63-67, :1523-1558; หน้าจอมีข้อความเตือนรายชื่อคอลัมน์ที่ขาด ui/src/pages/Ingestion.jsx:865	ผู้ใช้ที่ชื่อคอลัมน์ต่างออกไป (เช่น marks) ไขกฎไม่ได้ และไม่มีทางแก้ นอกจากแก้ไฟล์	ตารางจับคู่ "คอลัมน์ที่กฎต้องการ ↔ คอลัมน์ในไฟล์" ให้เลือกคอลัมน์เองจากรายการ พร้อมเครื่องหมายผ่าน/ไม่ผ่านรายคอลัมน์ ส่งการจับคู่ไปกับการอัปโหลด	D + F	M	api/app/api/whitebox.py, ui/src/pages/Ingestion.jsx
BB-02	ไฟล์ที่ชื่อคอลัมน์ไม่ตรงถูก "รับเข้า" โดยเปลี่ยนชื่อชุดข้อมูลเป็นชื่อไฟล์ใหม่ แต่ตัวเลขที่ตอบกลับยังคำนวณจากชุดข้อมูลเดิม	ตั้งชื่อใหม่ที่ api/app/api/whitebox.py:1519-1520 แล้วตอบ _recompute_interactive_state() ที่อ่านไฟล์เดิมที่ :1557 (อนุมานจากโค้ด ไม่ได้ทดสอบ เพราะรายงานนี้ห้ามอัปโหลด)	หน้าจอเห็นชื่อไฟล์ใหม่คู่กับตัวเลขของไฟล์เก่า	ผูกทุกผลลัพธ์กับรหัสชุดข้อมูล (แฮชไฟล์ + จำนวนแถว) ที่ระบบคำนวณเองและแสดงคู่กับตัวเลข; ไม่เปลี่ยนชื่อชุดข้อมูลจนกว่าเอนจินจะรันกับไฟล์นั้นจริง	E + F	S	api/app/api/whitebox.py
BB-03	ปุ่มเชื่อมต่อฐานข้อมูล / REST / Kafka ไม่ได้เชื่อมต่อจริง แค่นับทีกชื่อและที่อยู่แล้วสรุปสถิติจากไฟล์ในเครื่อง	api/app/api/whitebox.py:1561-1583 (rows_ingested ค่าเริ่มต้น 10100 ที่ :1580); ฟอรัม ui/src/pages/Ingestion.jsx:706, :747, :784	ผู้ใช้รอกที่อยู่แล้วเห็นผล "สำเร็จ" ทั้งที่ไม่มีการเชื่อมต่อ	ปิดใช้งานปุ่มที่ไม่ได้ต่อจริง (หรือให้ปุ่มตอบด้วยผลการเชื่อมต่อจริงเท่านั้น)	F	S	ui/src/pages/Ingestion.jsx, api/app/api/whitebox.py
BB-04	การ์ดสรุปข้อมูลใช้ค่าตายตัวที่ไม่ตรงกับกฎ: ตัวคูณ 1.5, คีย์ซ้ำ student_id + course + semester, ช่วงคะแนน 0-100 เฉพาะคอลัมน์ชื่อ score	api/app/api/whitebox.py:258-259, :281-286, :301-305; หน้าจอยอมรับเป็นข้อความ ui/src/pages/Ingestion.jsx:921	ตัวเลข "ค่าผิดปกติ" บนหน้านี้กับผลของกฎ (ตัวคูณ 3.0 ค่าเริ่มต้น) ค่าวนเคนละสูตร	คำนวณสรุปด้วยพารามิเตอร์ชุดเดียวกับกฎ (ค่า k, คีย์, ช่วงคะแนนที่ตั้งไว้) จึงเป็นตัวเลขเดียวทุกหน้า	F + A	M	api/app/api/whitebox.py (_compute_profile)
BB-05	อัปโหลดไฟล์ใหม่ทับชุดข้อมูลเดิมทันทีที่ไม่มีรายการชุดที่เคยนำเข้า	api/app/api/whitebox.py:1526 (เขียนทับ DIRECTORY_DATASET_PATH)	ผู้ใช้ไม่เห็นว่ามีชุดเก่าหายไป แล้วผลของกฎถูกผูกกับชุดไหน	รายการชุดข้อมูลที่นำเข้า (ชื่อ / เวลา / จำนวนแถว / แฮช) เลือกกลับไปใช้ได้	E	L	api/app/api/whitebox.py, ui/src/pages/Ingestion.jsx

Expectations & Alerts: กฎ 3 การ์ด

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประ เภ ท	ข น า ด	ไฟล์ที่ต้องแก้
B B- 06	ทุกการแก้กับการดมีผลทันที ไม่มีสถานะ "ร่าง" และปุ่ม "บันทึกกฎ" แคล่งค่าเดิมซ้ำ	ui/src/pages/RulesConfig.jsx:130-149, :191-206, :645, :774; api/app/api/whitebox.py:1418-1444	ผู้ใช้ไม่รู้วาค่าที่พิมพ์ "ใช่แล้ว" หรือยัง และไม่เห็นผลต่างก่อนใช้	สถานะ ร่าง / ใช่จริง: เปลี่ยนค่าแล้วเห็นตัวอย่างผลต่าง (แถวที่ย้ายโซน) ก่อน กด "ใช่" จึงเขียนสถานะและได้เลขเวอร์ชันของชุดกฎ	C + E	M	ui/src/pages/RulesConfig.jsx, api/app/api/whitebox.py (endpoint ประเมินผลแบบไม่บันทึก)
B B- 07	ปุ่มบันทึกแสดงผล "สำเร็จ" แม้ api ตอบผิดพลาด	ui/src/pages/RulesConfig.jsx:156-159 (กลืนข้อผิดพลาดคืน null), :194-200 (แสดงสำเร็จเสมอ)	ผลบนจอไม่ตรงกับสิ่งที่เกิดขึ้นจริง	ผูกสถานะปุ่มกับผลตอบกลับจริง: สำเร็จเมื่อได้ค่ากลับมา สัมเหลวให้สถานะสีแดงพร้อมรหัสตอบกลับ	F	S	ui/src/pages/RulesConfig.jsx
B B- 08	วงกลมเปอร์เซ็นต์บนการ์ดเป็น 95 / 99 / 99 ดายตัว ขณะที่ตารางสรุปใต้การ์ดคำนวณจริงได้ 93.7 / 97.1 / 98.1	ui/src/pages/RulesConfig.jsx:631, :693, :741 เทียบ :602-603, :805-820	สองตัวเลขสำหรับกฎเดียวกันบนหน้าเดียว	ให้วงกลมใช้ตัวเลขเดียวกับตารางสรุป (หรือเอาวงกลมออก)	F	S	ui/src/pages/RulesConfig.jsx
B B- 09	ตัวคูณ k: ป้าย > 12.0h / > 9.0h ดายตัว ขณะที่รั้วจริงของไฟล์ทดสอบคือ 23.0 / 15.5 และเปลี่ยน k ไม่เปลี่ยนแถวใดเลย (ทั้งสองค่าจับ 20 แถวเท่ากัน) ทั้งรั้วล่างก็ไม่แสดง	ui/src/pages/RulesConfig.jsx:772-779, :740; api/app/api/whitebox.py:1305-1313; S1, S2	ผู้ใช้ไม่มีทางเห็นว่า k ทำอะไร และป้ายบอกตัวเลขผิด	กราฟการกระจาย study_hours จากไฟล์จริง พร้อมเส้น Q1, Q3, รันบน และรั้วล่างที่เลื่อนตาม k และตัวเลขนับจำนวนแถวในแต่ละฝั่ง เห็นได้ทันทีว่ารั้ว 23.0 กับ 15.5 จับแถวชุดเดียวกัน	A	M	api/app/api/whitebox.py (endpoint ข้อมูลกราฟ), ui/src/pages/RulesConfig.jsx
B B- 10	ลำดับการ์ด 1→2→3 ไม่ตรงกับลำดับตรวจจริง (ซ้ำ → วาง/นอกช่วง → ค่าผิดปกติ) และชื่อ Gate 1/Gate 2 สลับกับลำดับจริง	api/app/api/whitebox.py:1291, :1294, :1312; ui/src/pages/Pipeline.jsx:279-280	จำนวน "Gate 1 กักกัน" คำนวณบนแถวที่ไม่ซ้ำ แต่แสดงก่อนซ้ำ ผู้ใช้เข้าใจผิดว่าตรวจอะไรก่อน	แผนภาพไหล (funnel/sankey) เรียงตามลำดับจริง แต่ละช่วงเป็นจำนวนแถวที่ถูกตัดออก กดแล้วกรองแถวนั้น ใช้ชุดเดียวกันทั้งหน้า Rule และ Pipeline	A + B	M	ui/src/pages/Pipeline.jsx, ui/src/pages/RulesConfig.jsx
B B- 11	ตัวเลข "แถวที่ติด" บนหน้า Rule กดแล้วไม่เห็นแถว	ui/src/pages/RulesConfig.jsx:648, :710, :758, :805-820 (มีแต่ตัวเลข); เทียบหน้า Ingestion ที่มีปุ่มดูตัวอย่าง 5 แถว ui/src/pages/Ingestion.jsx:138-152	ตามรอยจากกฎไปถึงแถวข้อมูลไม่ได้บนหน้าที่ตั้งกฎเอง	กดตัวเลขแล้วเจาะลงไปดูแถวที่กฎนั้นจับ พร้อมไฮไลต์คอลัมน์ที่ผิด (ใช้ preview-zone ที่มีอยู่แล้ว api/app/api/whitebox.py:1466-1496)	B	S	ui/src/pages/RulesConfig.jsx
B B- 12	เลือกคีย์ที่ไม่มีในไฟล์ (ตัวเลือก record_id) แล้วเอนจินใช้คีย์เดิมเจ๊บบๆ แต่ป้ายเขียนชื่อคีย์ที่เลือก	ui/src/pages/RulesConfig.jsx:728; api/app/api/whitebox.py:1279-1281, :1322; S7	ป้ายเหตุผลบอกคีย์ที่ไม่ได้ถูกใช้จริง	ให้เลือกได้เฉพาะคอลัมน์ที่มีในไฟล์ และแสดง "คีย์ที่ใช้จริง" (รายการคอลัมน์) ที่เอนจินคืนมาเอง แทนสตริงที่ผู้ใช้เลือก	E + F	S	api/app/api/whitebox.py, ui/src/pages/RulesConfig.jsx

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประ เภ ภ ท	ข น า ด	ไฟล์ที่ต้องแก้
B B- 13	ไฟล์ที่ไม่มีคอลัมน์ semester แต่มีอีก 4 คอลัมน์ ทำให้คีย์ซ้ำหุดเหลือ 2 คอลัมน์เรียงๆ: "ซ้ำ" เพิ่มจาก 30 เป็น 213 แถว, Clean ลดจาก 915 เหลือ 746 แต่ป้ายยังเขียนคีย์ครบสามคอลัมน์	api/app/api/whitebox.py:1279-1281 (เก็บเฉพาะคอลัมน์ที่มี); S10	แถว 183 แถวที่เป็นนักเรียนคนเดิมเรียนวิชาเดิมคนละเทอมถูกกักกันเป็นแถวซ้ำโดยไม่มีสัญญาณเตือน	เช่นเดียวกับ BB-12: แสดงคีย์ที่ใช้จริง และให้ตัวอย่างผลก่อนใช้ (C) ที่เห็นว่า "ซ้ำ" เพิ่มจาก 30 เป็น 213 แถว (ประมาณ 7 เท่า)	E + C	S	api/app/api/whitebox.py
B B- 14	ตัวเลือกที่เอนจินรองรับแต่ไม่มีช่องตั้ง (null_policy = adaptive_5, dedup_strategy = review_all) และหน้าจอส่งค่าเริ่มต้นทับทุกครั้งที่แก้การ์ดอื่น	api/app/api/whitebox.py:1295-1296, :1319; ui/src/pages/RulesConfig.jsx:113-116, :135-138; S4-S6	ความสามารถที่ผู้ใช้ไม่เห็นและค่าที่ตั้งผ่าน API ถูกรีเซ็ตเจียบๆ	เปิดเป็นตัวเลือกบนการ์ด (พร้อมตัวอย่างผลของ C) หรือเอาออกจากเอนจิน	F	S	ui/src/pages/RulesConfig.jsx
B B- 15	สวิตช์เปิดปิดกฎมีสองชั้นอยู่คนละหน้า: สวิตช์บนการ์ดกับ selected_findings จากหน้า Data Ingestion	api/app/api/whitebox.py:1286-1289, :1243-1245; ui/src/pages/Ingestion.jsx:241-249	ปิดที่หน้า Ingestion แล้วการ์ดยังดูเปิดอยู่ กฎไม่ทำงานโดยผู้ใช้ไม่รู้	สวิตช์เดียวต่อกฎ หรือให้การ์ดแสดงสถานะรวม (เปิด/ปิดโดยหน้าไหน) ด้วยตัวบ่งชี้สี	F + E	S	ui/src/pages/RulesConfig.jsx, ui/src/pages/Ingestion.jsx
B B- 16	Q1/Q3 ของรั้วคำนวณจากทุกแถว รวมแถวซ้ำและแถวที่ถูกกักกันแล้ว ขณะที่ความคิดเห็นในโค้ดบอกว่าคำนวณเฉพาะแถวที่ผ่านด่านก่อนหน้า	api/app/api/whitebox.py:1300-1304	ผู้ใช้ไม่รู้ว่ามีข้อมูลกลุ่มไหนถูกใช้คำนวณรั้ว	กราฟใน BB-09 ระบุจำนวนแถวที่ใช้คำนวณ (n) และให้เลือกได้ว่าคิดจากกลุ่มไหน	A + E	S	api/app/api/whitebox.py

คำนวณด้วยมือจากสูตรที่ ui/src/pages/RulesConfig.jsx:602-603 (ตัวเลขใน BB-08) ผ่าน = (ทั้งหมด - ที่ติด) ÷ ทั้งหมด โดยจำนวนที่ติดมาจาก S1_default.metrics (gate1_quarantined = 65, gate2_quarantined = 30, initial_outlier_count = 20) ทั้งหมด = 1,030 แถว

กฎ 1: (1030 - 65) / 1030 * 100 = 965 / 1030 * 100 = 93.69 -> แสดง 93.7% (วงกลมตายตัว: 95)
กฎ 2: (1030 - 30) / 1030 * 100 = 1000 / 1030 * 100 = 97.09 -> แสดง 97.1% (วงกลมตายตัว: 99)
กฎ 3: (1030 - 20) / 1030 * 100 = 1010 / 1030 * 100 = 98.06 -> แสดง 98.1% (วงกลมตายตัว: 99)

Expectations & Alerts: ฎรยตารยง (Spark)

I D	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประ เภ ษ ท	ข น า ด	ไฟล์ที่ต้องแก้
B B- 1 7	เกณฑัคุณภาพที่ Spark ใช้จริงถูกบันทึก ถูกรอบ แต่หน้าเว็บไม่แสดงเลย ทั้ง eff ective_quality_threshold, va lue_range_profile, rules_mod e	spark/spark_quality_engine.py:2823-2830; endpoint ประวัติรอบคินเอกสารเดิมอยู่แล้ว api/app/api/quality.py:39- 49 แต่คัน ui/src ไม่พบการอ้างถึงไฟล์เหล่านี้	ผู้ใช้เห็นแต่ค่าที่ตั้ง ไว้ ไม่เห็นค่าที่ใช้ ตัดสินใจแต่ละรอบ	กราฟ 15 รอบล่าสุดของ quality_ score พร้อมเส้นเกณฑ์ที่ใช้จริง เส้น ค่าเฉลี่ย – ส่วนเบี่ยงเบน และเส้นชั้น ต่ำ min_value และแผนรย ละเอียดรอบที่แสดงไว้ IQR ของ แต่ละคอส้มน์	A + E	M	ui/src/pages/Pip eline.jsx (แผนรย ละเอียดรอบ) ไม่ต้องแก้ api เพราะข้อมูลมาถึง เบราร์เซอร์แล้ว
B B- 1 8	โหมด adaptive ของเกณฑัคุณภาพไม่ เคยทำงานจริง: ถูกรอบใช้ค่าฐาน 90.0 เพราะ spark-master ไม่มีตัวแปร E LASTICSEARCH_URL ที่ฟังก์ชัน คำนวณต้องใช้	spark/dynamic_rules_engine.py:76-83, :301-305; งาน ถูกส่งผ่านเดมอนใน spark-master api/app/api/pipeline. py:204-208 (ทางสำรองในคอนเทนเนอร์ api :221 ไม่ได้ใช้ตาม ผลรันที่บันทึก); หลักฐาน spark-adaptive-threshold-prob e.txt (กรณี A = 90.0, กรณี B = 100.0), spark-threshold-b y-table.json	ฟอร์มมีตัวเลือก "Adaptive Dynamic Bounds" แต่เลือก แล้วไม่มีผล และไม่มี อะไรนอกว่ไม่ ทำงาน	แก้ที่ต้นเหตุ (ส่งตัวแปรให้ตรงกัน) ก่อนแสดงกราฟใน BB-17 มีฉะนั้น กราฟจะเป็นเส้นตรงที่ 90 และต้อง ตัดสินใจสูตร (ดูหัวข้อ "ควรปรับ อะไร" ข้อ 1)	F	S	docker-compose.y ml (บริการ spark-m aster/spark-work er), spark/dynami c_rules_engine.p y
B B- 1 9	ไฟล์ rules_mode ในผลรันนอก โครงสร้างของดั่งค่า ไม่ใช่สิ่งที่เกิดขึ้น จริง: เป็น adaptive ถูกรอบทั้งที่ใช้ค่า ฐาน	spark/spark_quality_engine.py:2826; spark-quality -runs-gov_data.json	ผลรันบอกว่า adaptive แต่เกณฑ์ ไม่เคยปรับ	บันทึกแหล่งของเกณฑ์จริงต่อรอบ (b ase / adaptive / fallback พร้อมสาเหตุ) ใน manifest ของรอบ	E	S	spark/spark_qual ity_engine.py, s park/dynamic_rul es_engine.py
B B- 2 0	ช่องบนฟอร์มที่ตั้งแล้วไม่มีผลต่อการ ตัดสินใจ 5 ช่อง: โหมดค่าว่าง (null_ch ecks), โหมดและค่าความหน่วงข้อมูล, ชื่อโมเดล AI, และโหมด adaptive ของ เกณฑัคุณภาพ	ตาราง "ดั่งค่าไหนมีผลจริง" ใน 02-spark-table-rules.md#ดั่งค ่าไหนมีผลจริง; ui/src/pages/RulesConfig.jsx:938-955, :985-1030, :1035-1042, :1072-1079	ผู้ใช้เชื่อว่าดั่งค่าแล้ว มีผล	ปิดใช้งานหรือเอาช่องเหล่านี้ออก (ยกเว้นช่องที่ทำให้ทำงานจริงตาม การตัดสินใจ)	F	S	ui/src/pages/Rul esConfig.jsx
B B- 2 1	มีการตรวจที่ทำงานเสมอและควบคุมไม่ ได้ หรือไม่แสดงผล: ลบแถวซ้ำเจียงๆ ก่อนตรวจ (ไม่กักกัน ไม่นับในคะแนน), Z-score 3.0 ดยตัว, IQR ครอบคลุม คอส้มน์ id/ลำดับ, สวิตช์ *.enabled ไม่มีโค้ดอ่าน	spark/spark_quality_engine.py:2043-2060, :2274, : 2241-2242; spark-quality-runs-gov_data.json (รับของ id, ลำดับ); spark-quality-runs-grocery_sales.json (resolved_1_duplicates)	ผู้ใช้ไม่เห็นว่แถว หายไปหรือถูกกักกัน ด้วยเหตุผลที่ตั้งเอง ไม่ได้	แผน "การตรวจที่รันรอบนี้" แสดง จำนวนแถวต่อการตรวจ (รวมแถวซ้ำที่ ถูกลบ, Z-score, IQR) แต่ละรายการ กดยดูแถวได้ และทำให้สวิตช์ *. enabled ทำงานจริงหรือเอาออก	E + B + F	M	spark/spark_qual ity_engine.py, u i/src/pages/Pipe line.jsx
B B- 2 2	กดยบันทึกฟอร์มส่งกฎที่รวมค่าเริ่มต้นแล้ว ทั้งก่อน อาจคัดลอกค่าเริ่มต้นทั้งหมดไป เป็นของตารยงนั้นถาวร (อนุมาณจากโค้ด ไม่ได้ทดสอบ)	ui/src/pages/RulesConfig.jsx:464, :537-541; api/a pp/api/dynamic_rules.py:685-688	ผู้ใช้แก้ช่องเดียวแต่ ผลที่ถูกเขียนคือทั้ง ชุด	ก่อนบันทึกแสดงผลต่างของสิ่งที่จะ ถูกเขียนจริง (เฉพาะช่องที่เปลี่ยน) และส่งเฉพาะช่องที่เปลี่ยน	C	S	ui/src/pages/Rul esConfig.jsx

I D	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประเภท	ขนาด	ไฟล์ที่ต้องแก้
B B- 2 3	หน้าจอแสดงกฎที่ "ตั้งไว้" ไม่ใช่กฎที่ Spark โหลดไปใช้ในรอบล่าสุด และ Spark ให้ความสำคัญกับ ES ก่อนไฟล์ ถ้าแก้ไฟล์ตรงๆ จะไม่มีผล	api/app/api/dynamic_rules.py:613-634; spark/spark_quality_engine.py:1530-1532	แหล่งข้อมูลกฎสองแหล่งที่ไม่ตรงกันได้ โดยหน้าจอไม่บอก	บันทึกแหล่งที่โหลด (ES หรือไฟล์) และตัวแทนของกฎที่ใช้ (เช่น แสช) ในผลรัน แล้วแสดงคู่กับกฎที่ตั้งไว้	E	S	spark/spark_quality_engine.py, ui/src/pages/RuleConfig.jsx
B B- 2 4	ตัวช่วยวิเคราะห์ AI: ชื่อโมเดลบนฟอร์ม ไม่ใช่ตัวที่ถูกเรียก (ชื่อค่าเริ่มต้นถูกสลับเป็นอีกตัว) และวิธีที่ใช้ (โมเดลภาษา หรือกฎสำเร็จรูป) บันทึกที่ข้อเสนอ ไม่อยู่ในผลรัน	spark/ai_rule_advisor.py:181-204, :218, :228; spark/spark_quality_engine.py:2711	ผู้ใช้ไม่รู้ว่าจะเสนอมาจากโมเดลหรือจากกฎสำเร็จรูป	บันทึกวิธี/โมเดลที่ใช้จริงต่อรันลงใน manifest และแสดงเป็นตัวอย่างชี้ในแผงรอบ	E	S	spark/spark_quality_engine.py, spark/ai_rule_advisor.py

Jobs & Pipelines

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประเภท	ขนาด	ไฟล์ที่ต้องแก้
B B- 25	เหตุผลรายแถวเป็นข้อความก่อนเดียวไม่มีค่าจริง เช่น Value Range Check [0.0, 100.0] (ตารางแสดงสามคอลัมน์: โชน / ประเภทปัญหา / เหตุผล)	ui/src/pages/Pipeline.jsx:33-35; api/app/api/whitebox.py:1331; S1_default.examples (แถว #21 score = -5.0 แสดงเพียงช่วง)	ต้องเปิดแถวเทียบเองว่าค่าอะไรทำให้ติด	คอลัมน์แยก: กฎ / คอลัมน์ / ค่าจริง / เงื่อนไข / เกณฑ์ (เช่น score / -5.0 / < 0 / 0) ฝั่ง Spark มีค่าจริงอยู่ในข้อความแล้ว (spark/dynamic_rules_engine.py:550-559) ให้แตกออกเป็นช่องเดียวกัน	D	M	api/app/api/whitebox.py, ui/src/pages/Pipeline.jsx, spark/dynamic_rules_engine.py
B B- 26	การตัดสินใจของผู้ตรวจหับผลของกฎโดยไม่เหลือร่องรอยของเหตุผลเดิม	api/app/api/whitebox.py:1337-1360 (ป้ายเหตุผลถูกเขียนทับ)	ตอบไม่ได้ว่าแถวนี้ติดกฎอะไรก่อนถูกอนุมัติ และค่าเดิมก่อนแก้	เก็บคอลัมน์ original_status, original_rule และค่าเดิมของแถวที่ถูกแก้ แสดงแถวที่ถูกทับด้วยตัวบ่งชี้สี	D	S	api/app/api/whitebox.py, ui/src/pages/Pipeline.jsx
B B- 27	ปุ่ม "รัน Pipeline" ฝั่งโต้ตอบส่งค่าว่างไปคำนวณใหม่ ผลไม่ต่างจากเดิม ไม่มีใบบันทึกหรือการเทียบรอบก่อน	ui/src/pages/Pipeline.jsx:134-139; api/app/api/whitebox.py:1420-1444	ผู้ใช้กดแล้วไม่เห็นว่าจะเกิดอะไร ผลเหมือนเดิมทุกครั้ง	manifest ของรอบ (เวลา / เวอร์ชันชุดกฎ / ค่าที่ใช้ / แสชชุดข้อมูล / จำนวนแต่ละโชน) และแสดงส่วนต่างเทียบรอบก่อน	E	M	api/app/api/whitebox.py, ui/src/pages/Pipeline.jsx

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประเภท	ขนาด	ไฟล์ที่ต้องแก้
B-B-28	ตัวเลขสองแหล่งบนหน้าเดียว: ผลของเอนจินโต้ตอบ (ในหน่วยความจำ ไม่เก็บประวัติ) กับประวัติรอบ Spark (เก็บถาวร) พฤติกรรมต่างกัน (Review กับกักกันตรงๆ, แลวซ้ำกักกันกับลบทิ้ง, ตำแหน่งแม่นยำกับประมาณ)	ตาราง "เทียบกับกฎ 3 การ์ด" ใน 02-s park-table-rules.md; ui/src/pages/Pipeline.jsx:269-271	ผู้ใช้เอาตัวเลขสองชุดมาเทียบกันโดยไม่รู้ว่าคำนวณคนละวิธี	แยกสองเอนจินด้วยรูปแบบภาพ: ตัวบ่งชี้สี่ประจำเอนจินติดกับทุกตัวเลข และแผงคนละแผง ไม่ปะปนในตารางเดียว	E	S	ui/src/pages/Pipeline.jsx
B-B-29	Spark: เหตุผลกักกันรายรอบ (quarantine_bre kdown) แสดงเฉพาะบนแดชบอร์ด ไม่แสดงที่หน้า Jobs & Pipelines ที่ดูประวัติรอบ	ui/src/pages/Dashboard.jsx: 1627-1632; ui/src/pages/Pipeline.jsx (ปุ่ม "ดูข้อมูล" ui/src/components/RunRecordsPanel.jsx)	ดูแถวได้แต่ไม่เห็นสรุปเหตุผลรอบรอบในทีเดียวกัน	แผงรายละเอียดรอบ (ร่วมกับ BB-17) แสดงเหตุผลผลกักกัน กดแท่งแล้วกรองแถวในแผง "ดูข้อมูล"	B + A	S	ui/src/pages/Pipeline.jsx, ui/src/components/RunRecordsPanel.jsx

Workspace Exports

ID	จุดที่เป็น black box	หลักฐาน	ทำไมจึงเป็น black box	วิธีทำให้เป็น white box	ประเภท	ขนาด	ไฟล์ที่ต้องแก้
BB-30	ไฟล์ที่ดาวน์โหลดไม่บอกมาจากกฎชุดไหน มีเพียงสามคอลัมน์ whitebox_* ต่อแถว ไฟล์ถูกสร้างใหม่จากสถานะปัจจุบันทุกครั้งที่ดาวน์โหลด และชื่อไฟล์ฝั่งเซิร์ฟเวอร์คงที่ student_course_scores_* ไม่ว่าชุดไหน	api/app/api/whitebox.py:1315-1317, :1447-1463 (คำนวณใหม่ที่ :1449, ชื่อไฟล์ที่ :1453-1459); ชื่อไฟล์ฝั่งหน้าเว็บ ui/src/pages/DataExport.jsx:87	สองคนดาวน์โหลดคนละเวลาหลังมีคนแก้กฎ ได้ไฟล์ต่างกันโดยไม่มีอะไรบอก	แนบใบบันทึกการรัน (JSON) มากับไฟล์: เวอร์ชันชุดกฎ ค่าที่ใช้ เวลาแฮชชุดข้อมูล จำนวนแถวต่อโซน และตั้งชื่อไฟล์จากรหัสรอบ	E	S	api/app/api/whitebox.py, ui/src/pages/DataExport.jsx

ควรปรับอะไร (เรียงตามลำดับความสำคัญ)

การจัดลำดับใช้สามเกณฑ์: (1) ปิดจุด black box ได้กี่ข้อ (2) ตอบข้อ 2 และข้อ 3 ของเพื่อนตรงแคไหน คือ "ค่าถูกส่งไปไหน" และ "ตัวเลขถูกเอาไปคำนวณยังใง" (3) ขนาดงาน

ลำดับ	สิ่งที่ควรปรับ	ปิด BB	เหตุผลที่อยู่ลำดับนี้	ขนาด	ไฟล์ที่แตะ
1	ใบบันทึกการรัน (manifest) ของทั้งสองเอนจิน: บันทึกค่าที่ใช้จริง (เกณฑ์, แหล่งของเกณฑ์, ชุดกฎ, แฮชชุดข้อมูล, จำนวนต่อโซน, การตรวจที่รัน) แล้วแสดงเทียบกับค่าที่ตั้งไว้ ฝั่ง Spark เก็บส่วนใหญ่อยู่แล้ว จึงเป็นงานฝั่งหน้าจอเป็นหลัก	BB-02, 17, 19, 21, 23, 24, 27, 30 (และ 29 บางส่วน)	ปิดได้มากที่สุด และเป็นฐานของแทบทุกข้อ ต้องแก้ BB-18 ก่อนหรือพร้อมกัน มิฉะนั้นกราฟเกณฑ์เป็นเส้นตรงที่ 90	M-L	spark/spark_quality_engine.py, docker-compose.yml, api/app/api/whitebox.py, ui/src/pages/Pipeline.jsx

ลำดับ	สิ่งที่ควรปรับ	ปิด BB	เหตุผลที่อยู่ลำดับนี้	ขนาด	ไฟล์ที่แตะ
2	กราฟการกระจายพร้อมเส้นรั้วที่เลื่อนตาม k ได้ พร้อมนับจำนวนแถวแต่ละฝั่ง	BB-09, 16	เห็นชัดที่สุดว่าตัวเลขในกฎถูกเอาไปคำนวณอย่างไร (ข้อ 3 ของเพื่อน)	M	api/app/api/whitebox.py, ui/src/pages/RulesConfig.jsx
3	ร่าง → เห็นผลต่าง → ยืนยัน พร้อมเลขเวอร์ชันชุดกฎ	BB-06, 13, 14, 22	ตอบข้อ 2 ของเพื่อนตรงที่สุด ("ตั้งค่าแล้วเกิดอะไรขึ้น")	M	ui/src/pages/RulesConfig.jsx, api/app/api/whitebox.py
4	เหตุผลรายแถวแบบแยกช่อง + กดตัวเลขแล้วเห็นแถว	BB-11, 25, 26, 29	ตามรอยได้ตั้งแต่กฎไปถึงแถวข้อมูล	M	api/app/api/whitebox.py, ui/src/pages/Pipeline.jsx, ui/src/pages/RulesConfig.jsx
5	เอาของปลอมออก: วงกลม 95/99/99, ป้าย > 12.0h, ช่องที่ไม่มีผล 5 ช่อง, ปุ่มเชื่อมต่อที่ไม่ได้ต่อ, สถานะ "สำเร็จ" ที่ไม่ตรงความจริง	BB-03, 07, 08, 20	งานเล็ก ผลชัด และเป็นเงื่อนไขของ "ระบบไม่โกหก"	S	ui/src/pages/RulesConfig.jsx, ui/src/pages/Ingestion.jsx
6	แพนด้าและชุดข้อมูลที่ใช่จริง: แสดงคีย์ที่ใช่จริง, ผูกผลกับรหัสชุดข้อมูล, ตารางจับคู่คอลัมน์ตอนอัปโหลด	BB-01, 04, 12, 15	ตัด "ตัวเลขถูกแต่ป้ายผิด" ที่พิสูจน์แล้ว (S7, S10)	M	api/app/api/whitebox.py, ui/src/pages/Ingestion.jsx, ui/src/pages/RulesConfig.jsx
7	แผนภาพไหลตามลำดับตรวจจริง ใช้ชุดเดียวกันทั้งหน้า Rule และ Pipeline	BB-10, 28	แก้ความเข้าใจลำดับตรวจ	M	ui/src/pages/Pipeline.jsx, ui/src/pages/RulesConfig.jsx
8	รายการชุดข้อมูลที่นำเข้า เลือกกลับไปใช้ได้	BB-05	ต้องแก้สภาวะติดการเก็บไฟล์ จึงเป็นงานใหญ่และอยู่ท้าย	L	api/app/api/whitebox.py, ui/src/pages/Ingestion.jsx

ข้อควรตัดสินใจก่อนลงมือข้อ 1 (ผลต่อพฤติกรรม) การซ่อม BB-18 ทำให้เกณฑ์ adaptive เริ่มทำงานจริง และสูตรปัจจุบัน `max(ค่าเฉลี่ย - ส่วนเบี่ยงเบน, min_value)` มีพฤติกรรมที่ควรถามเพื่อนก่อน (ดูข้อ 3.1 ใน 02-spark-table-rules.md): เกณฑ์ใหม่ถูกจำกัดด้วยค่าฐาน (`gov_data` ได้ 100.0 แทน 90.0) และตารางที่ต่ำคงที่จะหยุดถูกเตือน จึงต้องเลือกว่าจะจำกัดเกณฑ์ไว้ที่ค่าฐาน หรือรับพฤติกรรมนี้

ควรเน้นอะไร

เมื่อนำเสนอ Transform ให้เน้น 5 ข้อนี้ ซึ่งมาจากสิ่งที่พิสูจน์ได้ในรายงานนี้:

- ตามรอยได้ตลอดสาย: คำกฎ → ตัวเลข → แถว → ไฟล์ (BB-06, 11, 25, 30) เป็นแกนของคำถามข้อ 2 และ 3 ของเพื่อน
- ตัวเลขหนึ่งตัว มีแหล่งเดียว วงกลม 95 เทียบตาราง 93.7 (BB-08), ป้าย 12.0h เทียบรั้วจริง 23.0h (BB-09), การ์ดสรุปตัวคุณ 1.5 เทียบกฎ 3.0 (BB-04) ต่างเป็นสองตัวเลขสำหรับเรื่องเดียวกัน
- ไม่มีช่องที่ตั้งแล้วไม่มีผล ฟอรัม Spark มี 8 ช่อง ใช้ตัดสินใจจริง 3 ช่อง (BB-20) และตัวเลือกที่มีจริงแต่ไม่มีช่อง (BB-14)
- ระบบสองเอนจินที่พฤติกรรมต่างกัน ต้องแสดงตัวตนของแต่ละอันจริงๆ (BB-28) ฟังก์ชันการส่งคำคิดปดไป Review ส่วน Spark กักกันจริงๆ ฟังก์ชันการตัดแถวเข้าเป็นกักกัน ส่วน Spark ลบทิ้ง
- บันทึกสิ่งที่รันจริง ไม่ใช่สิ่งที่ตั้งไว้ (BB-19) `rules_mode = adaptive` ทั้งที่ใช้ค่าฐาน คือตัวอย่างของระบบที่รายงานสิ่งที่ตั้งไว้แทนสิ่งที่เกิดขึ้น

ข้อที่ตรวจแล้วไม่จริง

ข้อสันนิษฐานที่ตั้งไว้ก่อนตรวจโค้ด แล้วพบว่าไม่จริง จึงไม่นำมาเป็นหัวข้อในตารางข้างบน:

1. "ตามรอยจากตัวเลขไปหาแถวไม่ได้เลย" ไม่จริงทั้งหมด หน้า Data Ingestion มีปุ่มดูตัวอย่าง 5 แถวต่อประเด็น (ui/src/pages/Ingestion.jsx:138-152) และ funnel บนหน้า Pipeline กดแล้วกรองโซนได้ (ui/src/pages/Pipeline.jsx:284-288) ที่ขาดคือหน้า Rule เอง (BB-11)
2. "หน้าเว็บไม่มีตัวเลขอัตราผ่านที่คำนวณจริง" ไม่จริง ตารางสรุปได้การ์ดคำนวณจริงจากผลรัน (ui/src/pages/RulesConfig.jsx:602-603) ปัญหาคือมัน ขัดกับวงกลมตายตัว บนหน้าเดียวกัน (BB-08)
3. "หน้า Data Ingestion ไม่บอกเลยว่าไฟล์ขาดคอลัมน์" ไม่จริง มีข้อความบอกรายชื่อคอลัมน์ที่ขาด (ui/src/pages/Ingestion.jsx:865) แต่เป็นข้อความ และไม่มีทางแก้ในหน้านั้น (BB-01)

แก้ไขจากคำตอบในแชทก่อนหน้านี้

คำตอบก่อนหน้านี้ในแชทมีข้อที่ตรวจโค้ดแล้วไม่ตรงกับความจริง ดังนี้ (สองข้อแรกอยู่ใน 02-spark-table-rules.md ด้วย)

1. เคยตอบว่า เกณฑ์ที่ Spark ใช้จริง "ไม่ได้บันทึกไว้" **ความจริง:** บันทึกทุกรอบ (spark/spark_quality_engine.py:2823-2830) จุดบอดคือหน้าเว็บไม่แสดง (BB-17)
2. เคยตอบว่า ตัวตรวจ value_range แบบ adaptive ตั้งแล้วไม่มีผล **ความจริง:** ตัวตรวจ IQR ทำงานจริง ช่องที่ไม่มีผลคือ null_checks
3. เคยตอบว่า ระบบคำนวณเกณฑ์ adaptive ใหม่ทุกรอบจากประวัติ 15 รอบ **ความจริง:** โค้ดเขียนไว้แบบนั้น แต่ในระบบที่ deploy อยู่ ไม่เคยทำงาน ทุกรอบใช้ค่าฐาน (BB-18)
4. เคยเสนอให้ เปลี่ยนวงกลม 95/99/99 เป็น "% ที่ผ่านกฎข้อนั้นจริง" เหมือนยังไม่มีตัวเลขจริงในหน้า **ความจริง:** มีตัวเลขจริงในตารางได้การ์ดแล้ว งานคือทำให้ตรงกัน ไม่ใช่สร้างใหม่ (BB-08)
5. เคยเสนอให้ เพิ่มการเจาะดูแถวจากตัวเลข เหมือนทั้งระบบไม่มี **ความจริง:** มีอยู่ที่หน้า Ingestion และ funnel ที่หน้า Pipeline งานที่เหลือคือหน้า Rule (BB-11)